

Context-Free Grammars

Derivations, Normal Forms, Properties, and Pumping

Computer Science Theory

Slide Set 5

Learning Outcomes

By the end of this slide set, you should be able to:

- define a context-free grammar (CFG) and the language it generates;
- distinguish sentential forms, sentences, and leftmost/rightmost derivations;
- prove that a proposed grammar generates exactly a specified language;
- define complete and partial derivation trees and compute their frontiers;
- demonstrate ambiguity using parse trees or leftmost derivations;
- recognize s -grammars and explain their predictive behavior;
- eliminate null, unit, and useless productions from a CFG;
- convert grammars to CNF and GNF and distinguish the two forms;
- apply CYK, analyze its complexity, and distinguish major extensions from alternative parsing algorithms;
- state the principal closure and decision properties of CFLs;
- use the CFL pumping lemma to prove languages non-context-free;
- rewrite grammars for precedence and express syntax using BNF/EBNF.

Road Map

- 1 Motivation: Recursive Structure
- 2 Definition and Derivations
- 3 Parse Trees
- 4 Ambiguity
- 5 Simple Grammars (s-Grammars)
- 6 Normal Forms for CFGs
- 7 Properties of Context-Free Languages
- 8 The Pumping Lemma for CFLs
- 9 Applications and Grammar Notation
- 10 Review and Exercises

Why Context-Free Grammars?

Finite automata recognize patterns using only finite memory. They cannot enforce an unbounded exact match such as

$$\{0^n 1^n \mid n \geq 0\}.$$

A CFG describes a language *generatively*: it recursively builds valid strings from a start variable.

What recursion gives us

A finite set of productions can describe arbitrarily deep nesting, matched delimiters, recursive program syntax, and self-similar structure.

Power and limitation

Every regular language is context-free, but some languages—for example $\{a^n b^n c^n \mid n \geq 0\}$ —are not context-free.

Why Context-Free Grammars?

Finite automata recognize patterns using only finite memory. They cannot enforce an unbounded exact match such as

$$\{0^n 1^n \mid n \geq 0\}.$$

A CFG describes a language *generatively*: it recursively builds valid strings from a start variable.

What recursion gives us

A finite set of productions can describe arbitrarily deep nesting, matched delimiters, recursive program syntax, and self-similar structure.

Power and limitation

Every regular language is context-free, but some languages—for example $\{a^n b^n c^n \mid n \geq 0\}$ —are not context-free.

Why Context-Free Grammars?

Finite automata recognize patterns using only finite memory. They cannot enforce an unbounded exact match such as

$$\{0^n 1^n \mid n \geq 0\}.$$

A CFG describes a language *generatively*: it recursively builds valid strings from a start variable.

What recursion gives us

A finite set of productions can describe arbitrarily deep nesting, matched delimiters, recursive program syntax, and self-similar structure.

Power and limitation

Every regular language is context-free, but some languages—for example $\{a^n b^n c^n \mid n \geq 0\}$ —are not context-free.

An Inductive Definition: Balanced Parentheses

A balanced-parenthesis string can be defined recursively:

- 1 the empty string is balanced;
- 2 if x and y are balanced, then $(x)y$ is balanced; and
- 3 nothing else is balanced.

This becomes the grammar

$$S \rightarrow (S)S \mid \varepsilon.$$

Examples

$\varepsilon, \quad (), \quad (()), \quad ()(), \quad (())()$

The first S creates nesting inside a pair; the second permits another balanced string to follow.

An Inductive Definition: Balanced Parentheses

A balanced-parenthesis string can be defined recursively:

- 1 the empty string is balanced;
- 2 if x and y are balanced, then $(x)y$ is balanced; and
- 3 nothing else is balanced.

This becomes the grammar

$$S \rightarrow (S)S \mid \varepsilon.$$

Examples

$\varepsilon, \quad (), \quad (()), \quad ()(), \quad (())()$

The first S creates nesting inside a pair; the second permits another balanced string to follow.

An Inductive Definition: Balanced Parentheses

A balanced-parenthesis string can be defined recursively:

- 1 the empty string is balanced;
- 2 if x and y are balanced, then $(x)y$ is balanced; and
- 3 nothing else is balanced.

This becomes the grammar

$$S \rightarrow (S)S \mid \varepsilon.$$

Examples

$\varepsilon, \quad (), \quad (()), \quad ()(), \quad (()())$

The first S creates nesting inside a pair; the second permits another balanced string to follow.

Formal Definition of a CFG

We use the Linz-style notation $G = (V, T, S, P)$. Sipser commonly writes (V, Σ, R, S) ; the components are the same up to names and order.

Definition

A context-free grammar is a 4-tuple $G = (V, T, S, P)$, where:

- 1 V is a finite set of variables (nonterminals);
- 2 T is a finite set of terminals, with $V \cap T = \emptyset$;
- 3 $S \in V$ is the start variable; and
- 4 P is a finite set of productions $A \rightarrow \alpha$, where $A \in V$ and $\alpha \in (V \cup T)^*$.

Alternatives such as $A \rightarrow x$, $A \rightarrow y$, and $A \rightarrow z$ are abbreviated $A \rightarrow x \mid y \mid z$.

Formal Definition of a CFG

We use the Linz-style notation $G = (V, T, S, P)$. Sipser commonly writes (V, Σ, R, S) ; the components are the same up to names and order.

Definition

A context-free grammar is a 4-tuple $G = (V, T, S, P)$, where:

- 1 V is a finite set of variables (nonterminals);
- 2 T is a finite set of terminals, with $V \cap T = \emptyset$;
- 3 $S \in V$ is the start variable; and
- 4 P is a finite set of productions $A \rightarrow \alpha$, where $A \in V$ and $\alpha \in (V \cup T)^*$.

Alternatives such as $A \rightarrow x$, $A \rightarrow y$, and $A \rightarrow z$ are abbreviated $A \rightarrow x \mid y \mid z$.

What Does “Context-Free” Mean?

A production has exactly one variable on its left-hand side:

$$A \rightarrow \alpha.$$

Therefore A may be replaced by α regardless of the symbols surrounding it.

Valid CFG productions	Not CFG productions
$S \rightarrow 0S1$	$AB \rightarrow BA$ (two variables on the left)
$A \rightarrow aB \mid \varepsilon$	$aA \rightarrow Aa$ (a terminal occurs on the left)
$B \rightarrow C$	$\varepsilon \rightarrow S$ (no variable on the left)

Terminology

“Context-free” describes the form of the productions. It does not mean that the generated strings lack structure or context.

What Does “Context-Free” Mean?

A production has exactly one variable on its left-hand side:

$$A \rightarrow \alpha.$$

Therefore A may be replaced by α regardless of the symbols surrounding it.

Valid CFG productions	Not CFG productions
$S \rightarrow 0S1$	$AB \rightarrow BA$ (two variables on the left)
$A \rightarrow aB \mid \varepsilon$	$aA \rightarrow Aa$ (a terminal occurs on the left)
$B \rightarrow C$	$\varepsilon \rightarrow S$ (no variable on the left)

Terminology

“Context-free” describes the form of the productions. It does not mean that the generated strings lack structure or context.

Derivations and Sentential Forms

If $A \rightarrow \alpha$ is a production, then for any $u, v \in (V \cup T)^*$,

$$uAv \Rightarrow u\alpha v.$$

- $\Rightarrow$: one derivation step;
- $\Rightarrow^*$: zero or more steps;
- $\Rightarrow^+$: one or more steps;
- **sentential form**: any γ such that $S \Rightarrow^* \gamma$;
- **sentence**: a terminal-only sentential form $w \in T^*$.

The generated language is

$$L(G) = \{w \in T^* \mid S \Rightarrow^* w\}.$$

Because $\Rightarrow^*$ allows zero steps, S itself is a sentential form but normally not a sentence.

Derivations and Sentential Forms

If $A \rightarrow \alpha$ is a production, then for any $u, v \in (V \cup T)^*$,

$$uAv \Rightarrow u\alpha v.$$

- $\Rightarrow$: one derivation step;
- $\Rightarrow^*$: zero or more steps;
- $\Rightarrow^+$: one or more steps;
- **sentential form**: any γ such that $S \Rightarrow^* \gamma$;
- **sentence**: a terminal-only sentential form $w \in T^*$.

The generated language is

$$L(G) = \{w \in T^* \mid S \Rightarrow^* w\}.$$

Because $\Rightarrow^*$ allows zero steps, S itself is a sentential form but normally not a sentence.

Leftmost and Rightmost Derivations

Consider

$$S \rightarrow AB, \quad A \rightarrow aA \mid a, \quad B \rightarrow bB \mid b.$$

A leftmost derivation of *aabb* is

$$S \Rightarrow AB \Rightarrow aAB \Rightarrow aaB \Rightarrow aabB \Rightarrow aabb.$$

A rightmost derivation of the same string is

$$S \Rightarrow AB \Rightarrow AbB \Rightarrow Abb \Rightarrow aAbb \Rightarrow aabb.$$

Key point

The replacement order differs, but both derivations describe the same hierarchical parse tree. Derivation order alone is not semantic structure.

Leftmost and Rightmost Derivations

Consider

$$S \rightarrow AB, \quad A \rightarrow aA \mid a, \quad B \rightarrow bB \mid b.$$

A leftmost derivation of *aabb* is

$$S \Rightarrow AB \Rightarrow aAB \Rightarrow aaB \Rightarrow aabB \Rightarrow aabb.$$

A rightmost derivation of the same string is

$$S \Rightarrow AB \Rightarrow AbB \Rightarrow Abb \Rightarrow aAbb \Rightarrow aabb.$$

Key point

The replacement order differs, but both derivations describe the same hierarchical parse tree. Derivation order alone is not semantic structure.

Proving What a Grammar Generates

For

$$G : S \rightarrow 0S1 \mid \varepsilon,$$

prove $L(G) = \{0^n 1^n \mid n \geq 0\}$ using two inclusions.

Soundness: $L(G) \subseteq \{0^n 1^n\}$

Every use of $S \rightarrow 0S1$ adds one 0 to the left and one 1 to the right. After k such steps the form is $0^k S 1^k$; replacing S by ε yields $0^k 1^k$.

Completeness: $\{0^n 1^n\} \subseteq L(G)$

For any $n \geq 0$, apply $S \rightarrow 0S1$ exactly n times and then $S \rightarrow \varepsilon$:
 $S \Rightarrow^* 0^n S 1^n \Rightarrow 0^n 1^n$.

Testing examples suggests a grammar; the two-inclusion argument proves it.

Proving What a Grammar Generates

For

$$G : S \rightarrow 0S1 \mid \varepsilon,$$

prove $L(G) = \{0^n 1^n \mid n \geq 0\}$ using two inclusions.

Soundness: $L(G) \subseteq \{0^n 1^n\}$

Every use of $S \rightarrow 0S1$ adds one 0 to the left and one 1 to the right. After k such steps the form is $0^k S 1^k$; replacing S by ε yields $0^k 1^k$.

Completeness: $\{0^n 1^n\} \subseteq L(G)$

For any $n \geq 0$, apply $S \rightarrow 0S1$ exactly n times and then $S \rightarrow \varepsilon$:
 $S \Rightarrow^* 0^n S 1^n \Rightarrow 0^n 1^n$.

Testing examples suggests a grammar; the two-inclusion argument proves it.

Reusable Grammar-Design Patterns I

Matched Blocks

$$S \rightarrow 0S1 \mid \varepsilon$$

Generates $0^n 1^n$ by adding matched symbols at opposite ends.

Palindromes

$$S \rightarrow 0S0 \mid 1S1 \mid 0 \mid 1 \mid \varepsilon$$

Add the same symbol to both ends; base cases determine parity.

Balanced Strings

$$S \rightarrow (S)S \mid \varepsilon$$

One variable position handles nesting and the other concatenation.

Informal Preview: One String, Two Structures

Before giving the formal definition, it is useful to recognize the problem informally. A grammar can sometimes generate the same terminal string with two genuinely different hierarchical meanings.

Arithmetic preview

The string

$\text{id} + \text{id} \times \text{id}$

might mean either

$(\text{id} + \text{id}) \times \text{id}$ or $\text{id} + (\text{id} \times \text{id})$.

This is called **structural ambiguity**: the sequence of terminals is fixed, but its parse-tree structure is not. Later we will define ambiguity formally and rewrite the grammar to encode precedence and associativity.

Why this preview matters

Earlier grammar constructions may offer several production choices. That is not automatically a problem; ambiguity occurs only when choices give two different structures for the *same* string.

Informal Preview: One String, Two Structures

Before giving the formal definition, it is useful to recognize the problem informally. A grammar can sometimes generate the same terminal string with two genuinely different hierarchical meanings.

Arithmetic preview

The string

$\text{id} + \text{id} \times \text{id}$

might mean either

$(\text{id} + \text{id}) \times \text{id}$ or $\text{id} + (\text{id} \times \text{id}).$

This is called **structural ambiguity**: the sequence of terminals is fixed, but its parse-tree structure is not. Later we will define ambiguity formally and rewrite the grammar to encode precedence and associativity.

Why this preview matters

Earlier grammar constructions may offer several production choices. That is not automatically a problem; ambiguity occurs only when choices give two different structures for the *same* string.

Informal Preview: One String, Two Structures

Before giving the formal definition, it is useful to recognize the problem informally. A grammar can sometimes generate the same terminal string with two genuinely different hierarchical meanings.

Arithmetic preview

The string

$\text{id} + \text{id} \times \text{id}$

might mean either

$(\text{id} + \text{id}) \times \text{id}$ or $\text{id} + (\text{id} \times \text{id})$.

This is called **structural ambiguity**: the sequence of terminals is fixed, but its parse-tree structure is not. Later we will define ambiguity formally and rewrite the grammar to encode precedence and associativity.

Why this preview matters

Earlier grammar constructions may offer several production choices. That is not automatically a problem; ambiguity occurs only when choices give two different structures for the *same* string.

Reusable Grammar-Design Patterns II

Suppose grammars with renamed, disjoint variables generate L_1 and L_2 from start variables S_1 and S_2 .

Goal	New start production	Meaning
$L_1 \cup L_2$	$S \rightarrow S_1 \mid S_2$	choose either grammar
$L_1 L_2$	$S \rightarrow S_1 S_2$	generate one string from each
L_1^*	$S \rightarrow S_1 S \mid \varepsilon$	repeat zero or more times

Language generation versus ambiguity

These constructions generate the intended languages, but they may produce ambiguous grammars when a string can be decomposed in more than one way.

Reusable Grammar-Design Patterns II

Suppose grammars with renamed, disjoint variables generate L_1 and L_2 from start variables S_1 and S_2 .

Goal	New start production	Meaning
$L_1 \cup L_2$	$S \rightarrow S_1 \mid S_2$	choose either grammar
$L_1 L_2$	$S \rightarrow S_1 S_2$	generate one string from each
L_1^*	$S \rightarrow S_1 S \mid \varepsilon$	repeat zero or more times

Language generation versus ambiguity

These constructions generate the intended languages, but they may produce ambiguous grammars when a string can be decomposed in more than one way.

A Grammar-Design Workflow

- 1 Identify the recursive or nested structure of valid strings.
- 2 Choose one variable for each meaningful syntactic category or phase.
- 3 Write recursive productions that preserve the required invariant.
- 4 Add base cases so the recursion can terminate.
- 5 Test the smallest valid strings and several invalid strings.
- 6 Prove both soundness and completeness.
- 7 Check whether the grammar accidentally introduces ambiguity.

Debugging question

If an invalid string is generated, which production allowed the invariant to break? If a valid string is missing, which necessary base case or recursive choice is absent?

A Grammar-Design Workflow

- 1 Identify the recursive or nested structure of valid strings.
- 2 Choose one variable for each meaningful syntactic category or phase.
- 3 Write recursive productions that preserve the required invariant.
- 4 Add base cases so the recursion can terminate.
- 5 Test the smallest valid strings and several invalid strings.
- 6 Prove both soundness and completeness.
- 7 Check whether the grammar accidentally introduces ambiguity.

Debugging question

If an invalid string is generated, which production allowed the invariant to break? If a valid string is missing, which necessary base case or recursive choice is absent?

Formal Definition of a Derivation Tree

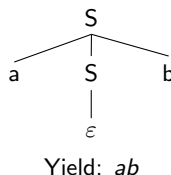
A **derivation tree** (parse tree) for $G = (V, T, S, P)$ is a finite, rooted, ordered tree satisfying:

- 1 the root is labeled S ;
- 2 every internal node is labeled by a variable;
- 3 if a node labeled A has children $X_1, \dots, X_k$ from left to right, then $A \rightarrow X_1 \cdots X_k \in P$; and
- 4 every leaf is labeled by a terminal or by ε .

The **yield** is obtained by reading the leaves from left to right and omitting ε .

Derivation-tree theorem

$S \Rightarrow^* w$ for $w \in T^*$ iff a derivation tree rooted at S has yield w .



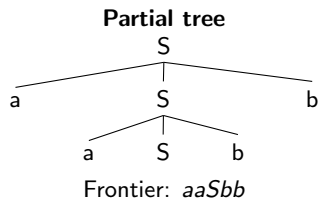
Partial Derivation Trees and Sentential Forms

A **partial derivation tree** obeys the same root and production conditions, but some leaves may still be variables. Its **frontier** is the left-to-right sequence of terminal and variable leaves, with ε omitted.

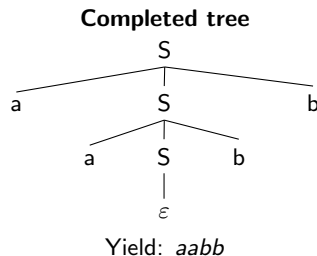
Formal correspondence

For every $\alpha \in (V \cup T)^*$,

$S \Rightarrow^* \alpha \iff$ some partial derivation tree has frontier α .



$\Rightarrow$



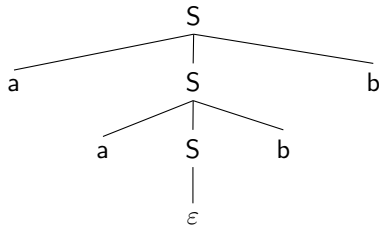
Worked Parse Tree for $aabb$

Grammar: $S \rightarrow aSb \mid \varepsilon$.

Derivation

$S \Rightarrow aSb \Rightarrow aaSbb \Rightarrow aabb$.

Each internal S node records one selected production.



Reading the leaves gives $aa\varepsilon bb$, whose yield is $aabb$.

What Is an Abstract Syntax Tree?

An **abstract syntax tree (AST)** is a compact tree representation of the essential constructs in a parsed input. Unlike a derivation tree, an AST is a compiler-design data structure rather than part of the formal definition of a CFG.

A parse tree retains

- grammar variables such as E , T , F ;
- punctuation and delimiters;
- every production application; and
- helper symbols introduced for parsing.

An AST retains

- operators and operands;
- declarations and statements;
- nesting and evaluation structure; and
- source locations needed for diagnostics.

Different concrete grammars can be translated into the same AST design, allowing later compiler phases to remain independent of parsing details.

What Is an Abstract Syntax Tree?

An **abstract syntax tree (AST)** is a compact tree representation of the essential constructs in a parsed input. Unlike a derivation tree, an AST is a compiler-design data structure rather than part of the formal definition of a CFG.

A parse tree retains

- grammar variables such as E , T , F ;
- punctuation and delimiters;
- every production application; and
- helper symbols introduced for parsing.

An AST retains

- operators and operands;
- declarations and statements;
- nesting and evaluation structure; and
- source locations needed for diagnostics.

Different concrete grammars can be translated into the same AST design, allowing later compiler phases to remain independent of parsing details.

What Is an Abstract Syntax Tree?

An **abstract syntax tree (AST)** is a compact tree representation of the essential constructs in a parsed input. Unlike a derivation tree, an AST is a compiler-design data structure rather than part of the formal definition of a CFG.

A parse tree retains

- grammar variables such as E , T , F ;
- punctuation and delimiters;
- every production application; and
- helper symbols introduced for parsing.

An AST retains

- operators and operands;
- declarations and statements;
- nesting and evaluation structure; and
- source locations needed for diagnostics.

Different concrete grammars can be translated into the same AST design, allowing later compiler phases to remain independent of parsing details.

Derivation Order Versus Tree Structure

Suppose a sentential form contains several variables. Expanding the left one first or the right one first may produce different step sequences but the same tree.

Correspondence theorem

Every parse tree determines exactly one leftmost derivation and exactly one rightmost derivation.

Consequently:

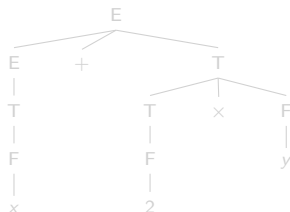
- two distinct arbitrary derivations do *not* necessarily prove ambiguity;
- two distinct leftmost derivations of the same string do prove ambiguity;
and
- two distinct parse trees of the same string are the clearest structural proof.

This distinction prevents one of the most common mistakes in CFG exercises.

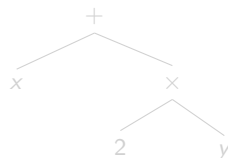
Worked AST Example: $x + 2 \times y$

Suppose the expression grammar enforces multiplication precedence over addition. The detailed parse tree and its AST record the same essential grouping at different abstraction levels.

Derivation/parse tree



Abstract syntax tree



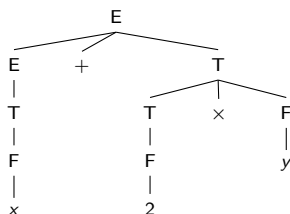
AST representation

The AST is $+(x, \times(2, y))$. Variables E, T, F disappear, while the operator hierarchy remains explicit for type checking, optimization, and code generation.

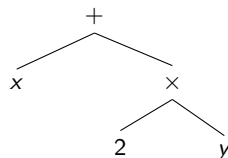
Worked AST Example: $x + 2 \times y$

Suppose the expression grammar enforces multiplication precedence over addition. The detailed parse tree and its AST record the same essential grouping at different abstraction levels.

Derivation/parse tree



Abstract syntax tree



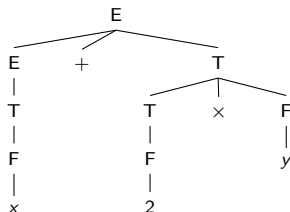
AST representation

The AST is $+(x, \times(2, y))$. Variables E, T, F disappear, while the operator hierarchy remains explicit for type checking, optimization, and code generation.

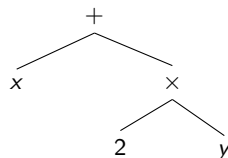
Worked AST Example: $x + 2 \times y$

Suppose the expression grammar enforces multiplication precedence over addition. The detailed parse tree and its AST record the same essential grouping at different abstraction levels.

Derivation/parse tree



Abstract syntax tree



AST representation

The AST is $+(x, \times(2, y))$. Variables E, T, F disappear, while the operator hierarchy remains explicit for type checking, optimization, and code generation.

What Is an Ambiguous Grammar?

Definition

A CFG is ambiguous if some string in its language has two distinct parse trees.

The following statements are equivalent:

- some string has two distinct parse trees;
- some string has two distinct leftmost derivations;
- some string has two distinct rightmost derivations.

Not enough

Two unrestricted derivations may simply expand independent variables in different orders and still describe one parse tree. Use parse trees or fixed-order derivations when proving ambiguity.

Lexical Meaning versus Structural Ambiguity

Natural language offers useful intuition, but distinguish two questions:

Lexical ambiguity: which meaning of a word?

"The bank was closed." The word *bank* could refer to a financial institution or a river bank (for example, one closed to visitors). Different word senses alone do not prove that a CFG has two parse trees.

Structural ambiguity: how are the words grouped?

"I saw the man with the telescope." Even with fixed word senses, *with the telescope* can describe the seeing event or the man. A grammar permitting both attachments gives two structures for one string.

The formal test is about a specified grammar

Two distinct parse trees suffice, whether or not a later interpretation gives them different values. Conversely, two possible meanings do not by themselves establish grammatical ambiguity.

For compiler CFGs, hold the token sequence fixed. Tokenization and semantic interpretation are separate questions.

Lexical Meaning versus Structural Ambiguity

Natural language offers useful intuition, but distinguish two questions:

Lexical ambiguity: which meaning of a word?

"The bank was closed." The word *bank* could refer to a financial institution or a river bank (for example, one closed to visitors). Different word senses alone do not prove that a CFG has two parse trees.

Structural ambiguity: how are the words grouped?

"I saw the man with the telescope." Even with fixed word senses, *with the telescope* can describe the seeing event or the man. A grammar permitting both attachments gives two structures for one string.

The formal test is about a specified grammar

Two distinct parse trees suffice, whether or not a later interpretation gives them different values. Conversely, two possible meanings do not by themselves establish grammatical ambiguity.

For compiler CFGs, hold the token sequence fixed. Tokenization and semantic interpretation are separate questions.

Lexical Meaning versus Structural Ambiguity

Natural language offers useful intuition, but distinguish two questions:

Lexical ambiguity: which meaning of a word?

"The bank was closed." The word *bank* could refer to a financial institution or a river bank (for example, one closed to visitors). Different word senses alone do not prove that a CFG has two parse trees.

Structural ambiguity: how are the words grouped?

"I saw the man with the telescope." Even with fixed word senses, *with the telescope* can describe the seeing event or the man. A grammar permitting both attachments gives two structures for one string.

The formal test is about a specified grammar

Two distinct parse trees suffice, whether or not a later interpretation gives them different values. Conversely, two possible meanings do not by themselves establish grammatical ambiguity.

For compiler CFGs, hold the token sequence fixed. Tokenization and semantic interpretation are separate questions.

Attachment Ambiguity: Who Has the Telescope?

“I saw the man with the telescope.”

- **VP attachment:** I used the telescope to see the man.
- **NP attachment:** I saw the man who had the telescope.

A small illustrative CFG, with start variable S , permits both:

$$\begin{aligned} S &\rightarrow \text{NP VP}, \\ \text{VP} &\rightarrow \text{V NP} \mid \text{VP PP}, \\ \text{NP} &\rightarrow \text{I} \mid \text{Det N} \mid \text{NP PP}, & \text{PP} &\rightarrow \text{P NP}, \\ \text{Det} &\rightarrow \text{the}, & \text{N} &\rightarrow \text{man} \mid \text{telescope}, & \text{V} &\rightarrow \text{saw}, & \text{P} &\rightarrow \text{with}. \end{aligned}$$

NP = noun phrase; VP = verb phrase; PP = prepositional phrase; Det = determiner; N = noun; V = verb; P = preposition.

Words are terminals; the final punctuation is omitted. This toy grammar illustrates attachment, not the full grammar of English.

Further reading: Bird, Klein, and Loper, *Natural Language Processing with Python*, Chapter 8.

Attachment Ambiguity: Who Has the Telescope?

“I saw the man with the telescope.”

- **VP attachment:** I used the telescope to see the man.
- **NP attachment:** I saw the man who had the telescope.

A small illustrative CFG, with start variable S , permits both:

$$\begin{aligned} S &\rightarrow \text{NP VP}, \\ \text{VP} &\rightarrow \text{V NP} \mid \text{VP PP}, \\ \text{NP} &\rightarrow \text{I} \mid \text{Det N} \mid \text{NP PP}, & \text{PP} &\rightarrow \text{P NP}, \\ \text{Det} &\rightarrow \text{the}, & \text{N} &\rightarrow \text{man} \mid \text{telescope}, & \text{V} &\rightarrow \text{saw}, & \text{P} &\rightarrow \text{with}. \end{aligned}$$

NP = noun phrase; VP = verb phrase; PP = prepositional phrase; Det = determiner; N = noun; V = verb; P = preposition.

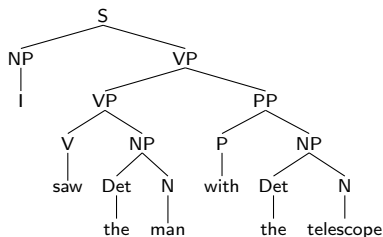
Words are terminals; the final punctuation is omitted. This toy grammar illustrates attachment, not the full grammar of English.

Further reading: [Bird, Klein, and Loper, *Natural Language Processing with Python*, Chapter 8.](#)

One Word Sequence, Two Attachment Trees

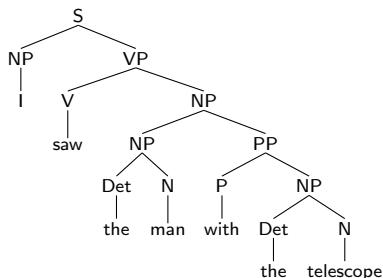
Both complete trees use the preceding CFG and have the identical yield: *I saw the man with the telescope.*

A: PP attaches to VP



The PP modifies the seeing event:
I used the telescope.

B: PP attaches to NP



The PP modifies the object:
the man had the telescope.

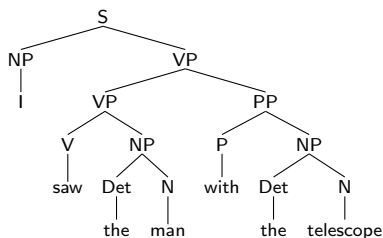
Bridge to the dangling else

Later, an else can attach to an inner or outer if. Both ask: *which constituent receives this attachment?* The telescope example is PP attachment, not a dangling modifier.

One Word Sequence, Two Attachment Trees

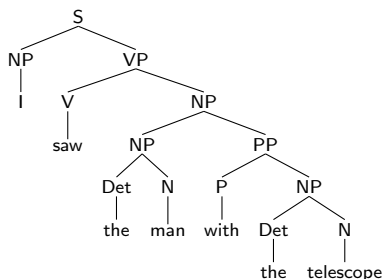
Both complete trees use the preceding CFG and have the identical yield: *I saw the man with the telescope.*

A: PP attaches to VP



The PP modifies the seeing event:
I used the telescope.

B: PP attaches to NP



The PP modifies the object:
the man had the telescope.

Bridge to the dangling else

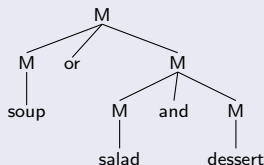
Later, an else can attach to an inner or outer if. Both ask: *which constituent receives this attachment?* The telescope example is PP attachment, not a dangling modifier.

Coordination Ambiguity: Is Dessert Always Included?

A restaurant advertises: **“Free soup or salad and dessert.”** Focus on the grouping of *soup or salad* and *dessert*; treat *or* as a choice and *and* as a bundle for this example.

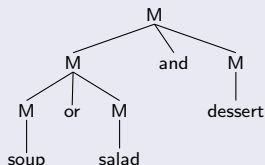
$$M \rightarrow M \text{ or } M \mid M \text{ and } M \mid \text{soup} \mid \text{salad} \mid \text{dessert}.$$

A: soup or (salad and dessert)



Dessert comes only with salad.
And groups more tightly here.

B: (soup or salad) and dessert



Dessert comes with either choice.
Or groups more tightly here.

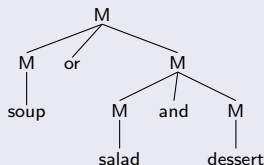
This is the same grouping issue as $5 + 3 \times 2$: $(5 + 3) \times 2 = 16$, while $5 + (3 \times 2) = 11$. The usual arithmetic convention selects 11; an ambiguous grammar alone does not enforce that convention. The next slides encode it explicitly.

Coordination Ambiguity: Is Dessert Always Included?

A restaurant advertises: **“Free soup or salad and dessert.”** Focus on the grouping of *soup or salad* and *dessert*; treat *or* as a choice and *and* as a bundle for this example.

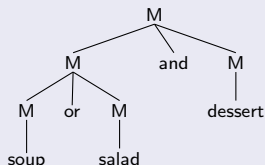
$$M \rightarrow M \text{ or } M \mid M \text{ and } M \mid \text{soup} \mid \text{salad} \mid \text{dessert}.$$

A: soup or (salad and dessert)



Dessert comes only with salad.
And groups more tightly here.

B: (soup or salad) and dessert



Dessert comes with either choice.
Or groups more tightly here.

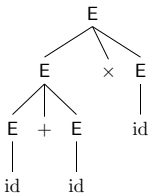
This is the same grouping issue as $5 + 3 \times 2$: $(5 + 3) \times 2 = 16$, while $5 + (3 \times 2) = 11$. The usual arithmetic convention selects 11; an ambiguous grammar alone does not enforce that convention. The next slides encode it explicitly.

Ambiguous Arithmetic Grammar

Consider

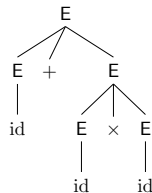
$$E \rightarrow E + E \mid E \times E \mid \text{id}.$$

The string $\text{id} + \text{id} \times \text{id}$ has two structures:



$(\text{id} + \text{id}) \times \text{id}$

$\text{id} + (\text{id} \times \text{id})$



The grammar does not encode precedence or associativity.

The Same Ambiguity as Leftmost Derivations

For $\text{id} + \text{id} \times \text{id}$:

Top-level multiplication

$$\begin{aligned} E &\Rightarrow_{\text{lm}} E \times E \Rightarrow_{\text{lm}} E + E \times E \Rightarrow_{\text{lm}} \text{id} + E \times E \\ &\Rightarrow_{\text{lm}} \text{id} + \text{id} \times E \Rightarrow_{\text{lm}} \text{id} + \text{id} \times \text{id}. \end{aligned}$$

Top-level addition

$$\begin{aligned} E &\Rightarrow_{\text{lm}} E + E \Rightarrow_{\text{lm}} \text{id} + E \Rightarrow_{\text{lm}} \text{id} + E \times E \\ &\Rightarrow_{\text{lm}} \text{id} + \text{id} \times E \Rightarrow_{\text{lm}} \text{id} + \text{id} \times \text{id}. \end{aligned}$$

Encoding Precedence and Associativity

Replace the ambiguous grammar by

$$E \rightarrow E + T \mid T,$$

$$T \rightarrow T \times F \mid F,$$

$$F \rightarrow (E) \mid \text{id}.$$

- Multiplication binds more tightly because it is formed inside T before T participates in addition.
- Left recursion in $E \rightarrow E + T$ makes addition left-associative.
- Left recursion in $T \rightarrow T \times F$ makes multiplication left-associative.
- Parentheses allow a factor to contain an entire expression.

For right-associative addition, one could instead use $E \rightarrow T + E \mid T$.

The Dangling-Else Ambiguity

Like the telescope's PP, the **else** has two possible attachment sites. The natural grammar

$$S \rightarrow \text{if } c \text{ then } S \mid \text{if } c \text{ then } S \text{ else } S \mid a$$

where c stands for a condition and a for any other statement. The grammar is ambiguous: in

$$\text{if } c \text{ then if } c \text{ then } a \text{ else } a,$$

the **else** may attach to either **if**.

Standard matched/unmatched solution

$$S \rightarrow M \mid U,$$

$$M \rightarrow a \mid \text{if } c \text{ then } M \text{ else } M,$$

$$U \rightarrow \text{if } c \text{ then } S \mid \text{if } c \text{ then } M \text{ else } U.$$

This grammar attaches each **else** to the nearest unmatched **if**.

Grammar Ambiguity Versus Inherent Ambiguity

- A **grammar** is ambiguous when it gives some string two parse trees.
- A **language** is inherently ambiguous when every CFG generating it is ambiguous.

An ambiguous grammar may still describe a language that has another unambiguous grammar; arithmetic expressions are an example.

Standard inherently ambiguous language

$$\{a^i b^j c^k \mid i = j \text{ or } j = k, i, j, k \geq 1\}.$$

Algorithmic limitation

There is no general algorithm that decides whether an arbitrary CFG is ambiguous.

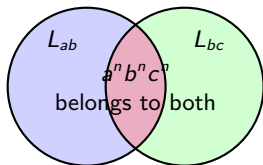
Understanding an Inherently Ambiguous Language

Write the standard example as a union:

$$L_{ab} = \{a^n b^n c^k : n, k \geq 1\},$$

$$L_{bc} = \{a^i b^n c^n : i, n \geq 1\},$$

$$L = L_{ab} \cup L_{bc}.$$



Strings $a^n b^n c^n$ have two natural structural explanations:

- match the a - and b -counts using L_{ab} ; or
- match the b - and c -counts using L_{bc} .

A direct union grammar therefore gives such strings two parse trees.

Why the theorem is stronger

Two parses in one convenient grammar prove only that grammar ambiguous. The inherent-ambiguity theorem proves that every CFG for L must be ambiguous; no grammar refactoring or normal-form conversion can remove the conflict. The proof that this is unavoidable is substantially deeper than displaying the two natural parses.

Definition of an s -Grammar

Simple grammar (Linz)

A CFG is an **s -grammar** if every production has the form

$$A \rightarrow a\alpha,$$

where $A \in V$, $a \in T$, and $\alpha \in V^*$, and for each pair (A, a) there is at most one such production.

Thus every right-hand side begins with a terminal, followed by zero or more variables. The uniqueness condition means that the current variable and the next input symbol determine at most one production.

Standard convention

The basic definition has no ϵ -productions, so an s -grammar does not generate ϵ . Some presentations add a separately controlled start rule for the empty word.

Definition of an s -Grammar

Simple grammar (Linz)

A CFG is an s -**grammar** if every production has the form

$$A \rightarrow a\alpha,$$

where $A \in V$, $a \in T$, and $\alpha \in V^*$, and for each pair (A, a) there is at most one such production.

Thus every right-hand side begins with a terminal, followed by zero or more variables. The uniqueness condition means that the current variable and the next input symbol determine at most one production.

Standard convention

The basic definition has no ε -productions, so an s -grammar does not generate ε . Some presentations add a separately controlled start rule for the empty word.

Worked *s*-Grammar Example

Consider

$$S \rightarrow aAB \mid bB, \quad A \rightarrow aA \mid b, \quad B \rightarrow b.$$

Every production begins with a terminal, and no variable has two productions beginning with the same terminal. Hence this is an *s*-grammar.

Leftmost derivation of *aabb*

$$S \Rightarrow aAB \Rightarrow aaAB \Rightarrow aabB \Rightarrow aabb.$$

At each step the next unmatched input symbol uniquely determines the production: (S, a) selects $S \rightarrow aAB$, (A, a) selects $A \rightarrow aA$, (A, b) selects $A \rightarrow b$, and (B, b) selects $B \rightarrow b$.

Predictive Recognition for an s-Grammar

A simple top-down recognizer stores the unexpanded variables, with the next variable at the left of the stack.

Input remaining	Variable stack	Action for the example grammar
<i>aabb</i>	<i>S</i>	use $S \rightarrow aAB$; match <i>a</i>
<i>abb</i>	<i>AB</i>	use $A \rightarrow aA$; match <i>a</i>
<i>bb</i>	<i>AB</i>	use $A \rightarrow b$; match <i>b</i>
<i>b</i>	<i>B</i>	use $B \rightarrow b$; match <i>b</i>
ϵ	ϵ	accept

Consequences

The production choice never requires backtracking. Therefore every s-grammar is unambiguous, although many unambiguous CFLs cannot be generated by an s-grammar.

What Fails the s-Grammar Test?

Valid

$$S \rightarrow aA \mid bB, \quad A \rightarrow a \mid bA.$$

For S , the leading terminals are distinct; the same is true for A .

Not simple

$$S \rightarrow aA \mid aB, \quad A \rightarrow b, \quad B \rightarrow c.$$

The pair (S, a) occurs twice, so lookahead a does not choose a unique production.

Rules such as $S \rightarrow AB$, $S \rightarrow \varepsilon$, or $S \rightarrow abA$ also violate the basic form: the first begins with a variable, the second has no leading terminal, and the third has a terminal after its first position.

Why Put a CFG in Normal Form?

A normal form restricts the shapes of productions without changing the generated language. The restricted structure makes proofs and algorithms easier to state and analyze.

Two standard forms

- **Chomsky normal form (CNF):** parse trees are binary, which supports CYK parsing and the standard pumping-lemma proof.
- **Greibach normal form (GNF):** every production emits a terminal first, closely connecting leftmost derivations with input consumption by a pushdown automaton.

Language equivalence, not identical grammar behavior

Conversion preserves the language (with a controlled exception for ϵ), but may change parse-tree shape, derivation length, and ambiguity behavior.

Why Put a CFG in Normal Form?

A normal form restricts the shapes of productions without changing the generated language. The restricted structure makes proofs and algorithms easier to state and analyze.

Two standard forms

- **Chomsky normal form (CNF):** parse trees are binary, which supports CYK parsing and the standard pumping-lemma proof.
- **Greibach normal form (GNF):** every production emits a terminal first, closely connecting leftmost derivations with input consumption by a pushdown automaton.

Language equivalence, not identical grammar behavior

Conversion preserves the language (with a controlled exception for ϵ), but may change parse-tree shape, derivation length, and ambiguity behavior.

Meaning of Equivalent Normal Form

Weak or language equivalence

Grammars G and G' are equivalent for this course when

$$L(G) = L(G').$$

CNF and GNF conversion guarantee this equality, subject to the explicit convention used for ε .

They do *not* generally preserve:

- the names or number of variables and productions;
- the exact derivation sequence or shape of the parse tree;
- the number of derivations associated with a word; or
- a convenient mapping to an intended abstract syntax tree.

Can every CFG be converted?

Every context-free language has a CNF grammar and a GNF grammar. If ε belongs to the language, both forms use the standard exceptional start rule $S_0 \rightarrow \varepsilon$, with S_0 absent from all right-hand sides.

The Empty String Requires Special Treatment

The rule $A \rightarrow \varepsilon$ can erase a variable inside a larger sentential form. This complicates binary parse-tree arguments and algorithms whose table cells represent *nonempty* input spans.

Standard segregation approach

Separate the empty word from the nonempty part of the language:

$$L = L^+ \cup \begin{cases} \{\varepsilon\}, & \varepsilon \in L, \\ \emptyset, & \varepsilon \notin L, \end{cases} \quad L^+ = L \setminus \{\varepsilon\}.$$

Introduce a fresh start variable S_0 that never occurs on a right-hand side. Preserve $S_0 \rightarrow \varepsilon$ when necessary, then eliminate every other null production.

Example

For $S \rightarrow aSb \mid \varepsilon$, use

$$S_0 \rightarrow S \mid \varepsilon, \quad S \rightarrow aSb \mid ab.$$

The second line generates the nonempty words; the exceptional start rule handles ε . A later unit-production step removes $S_0 \rightarrow S$.

CYK likewise handles an empty input by a separate test for $S_0 \rightarrow \varepsilon$.

The Empty String Requires Special Treatment

The rule $A \rightarrow \varepsilon$ can erase a variable inside a larger sentential form. This complicates binary parse-tree arguments and algorithms whose table cells represent *nonempty* input spans.

Standard segregation approach

Separate the empty word from the nonempty part of the language:

$$L = L^+ \cup \begin{cases} \{\varepsilon\}, & \varepsilon \in L, \\ \emptyset, & \varepsilon \notin L, \end{cases} \quad L^+ = L \setminus \{\varepsilon\}.$$

Introduce a fresh start variable S_0 that never occurs on a right-hand side. Preserve $S_0 \rightarrow \varepsilon$ when necessary, then eliminate every other null production.

Example

For $S \rightarrow aSb \mid \varepsilon$, use

$$S_0 \rightarrow S \mid \varepsilon, \quad S \rightarrow aSb \mid ab.$$

The second line generates the nonempty words; the exceptional start rule handles ε . A later unit-production step removes $S_0 \rightarrow S$.

CYK likewise handles an empty input by a separate test for $S_0 \rightarrow \varepsilon$.

The Empty String Requires Special Treatment

The rule $A \rightarrow \varepsilon$ can erase a variable inside a larger sentential form. This complicates binary parse-tree arguments and algorithms whose table cells represent *nonempty* input spans.

Standard segregation approach

Separate the empty word from the nonempty part of the language:

$$L = L^+ \cup \begin{cases} \{\varepsilon\}, & \varepsilon \in L, \\ \emptyset, & \varepsilon \notin L, \end{cases} \quad L^+ = L \setminus \{\varepsilon\}.$$

Introduce a fresh start variable S_0 that never occurs on a right-hand side. Preserve $S_0 \rightarrow \varepsilon$ when necessary, then eliminate every other null production.

Example

For $S \rightarrow aSb \mid \varepsilon$, use

$$S_0 \rightarrow S \mid \varepsilon, \quad S \rightarrow aSb \mid ab.$$

The second line generates the nonempty words; the exceptional start rule handles ε . A later unit-production step removes $S_0 \rightarrow S$.

CYK likewise handles an empty input by a separate test for $S_0 \rightarrow \varepsilon$.

Eliminating Null Productions

A **null production** is a rule $A \rightarrow \varepsilon$. A variable is **nullable** if it derives ε in zero or more steps.

Procedure

- 1 Mark every A for which $A \rightarrow \varepsilon$ is a production.
- 2 Repeatedly mark A when some rule $A \rightarrow X_1 \cdots X_k$ has every X_i already marked nullable.
- 3 For each production, add every nonempty alternative obtained by deleting any subset of its nullable-variable occurrences.
- 4 Remove the original null productions.
- 5 If $\varepsilon \in L(G)$ must be preserved, introduce a fresh start variable S_0 and retain $S_0 \rightarrow \varepsilon$.

Different subsets can produce the same rule; duplicate productions are kept only once.

Eliminating Null Productions

A **null production** is a rule $A \rightarrow \varepsilon$. A variable is **nullable** if it derives ε in zero or more steps.

Procedure

- 1 Mark every A for which $A \rightarrow \varepsilon$ is a production.
- 2 Repeatedly mark A when some rule $A \rightarrow X_1 \cdots X_k$ has every X_i already marked nullable.
- 3 For each production, add every nonempty alternative obtained by deleting any subset of its nullable-variable occurrences.
- 4 Remove the original null productions.
- 5 If $\varepsilon \in L(G)$ must be preserved, introduce a fresh start variable S_0 and retain $S_0 \rightarrow \varepsilon$.

Different subsets can produce the same rule; duplicate productions are kept only once.

Worked Null-Production Elimination

Consider

$$S \rightarrow AB \mid a, \quad A \rightarrow aA \mid \varepsilon, \quad B \rightarrow bB \mid \varepsilon.$$

Variables A and B are nullable directly, and S is nullable through $S \Rightarrow AB \Rightarrow^* \varepsilon$.

Grammar after null elimination

$$\begin{aligned} S_0 &\rightarrow S \mid \varepsilon, \\ S &\rightarrow AB \mid A \mid B \mid a, \\ A &\rightarrow aA \mid a, \\ B &\rightarrow bB \mid b. \end{aligned}$$

From $S \rightarrow AB$, deleting nullable occurrences gives $S \rightarrow A$, $S \rightarrow B$, and $S \rightarrow \varepsilon$; the last alternative is represented only by the fresh start rule. Similarly, $A \rightarrow aA$ yields $A \rightarrow a$ and $B \rightarrow bB$ yields $B \rightarrow b$.

Eliminating Unit Productions

A **unit production** has the form $A \rightarrow B$, with both sides variables.

Procedure

- 1 Compute all **unit pairs** (A, B) such that $A \Rightarrow^* B$ using only unit productions.
- 2 For every unit pair (A, B) , copy each non-unit production $B \rightarrow \alpha$ to $A \rightarrow \alpha$.
- 3 Delete all unit productions and remove duplicates.

Example

From

$$S \rightarrow A \mid b, \quad A \rightarrow B \mid a, \quad B \rightarrow bB \mid c,$$

the unit pairs include (S, A) , (S, B) , and (A, B) . The equivalent unit-free rules are

$$S \rightarrow b \mid a \mid bB \mid c, \quad A \rightarrow a \mid bB \mid c, \quad B \rightarrow bB \mid c.$$

Removing Useless Productions and Symbols

A variable is useful only if it can derive a terminal word and can occur in some derivation from the start variable.

Two-pass procedure

- 1 **Generating pass:** Mark variables with a terminal-only production, then repeatedly mark A if some $A \rightarrow \alpha$ contains only terminals and already generating variables. Remove everything involving an unmarked variable.
- 2 **Reachability pass:** Mark S . Whenever a marked variable has a production containing variable B , mark B . Remove all remaining unmarked variables and their productions.

For

$$S \rightarrow AB \mid a, \quad A \rightarrow aA \mid a, \quad B \rightarrow b, \quad C \rightarrow cC \mid c, \quad D \rightarrow dD,$$

D is non-generating and is removed first; C is generating but unreachable and is removed in the second pass. Only S, A, B remain.

Chomsky Normal Form (CNF)

Definition

A CFG is in Chomsky normal form when every production has one of the forms

$$A \rightarrow BC \quad \text{or} \quad A \rightarrow a,$$

where A, B, C are variables and a is a terminal. If $\varepsilon \in L(G)$, the additional rule $S_0 \rightarrow \varepsilon$ is allowed, provided the start variable S_0 never occurs on a right-hand side.

- Unit rules $A \rightarrow B$, mixed rules $A \rightarrow aB$, and right-hand sides of length greater than two are not in CNF.
- Every CFL has an equivalent CNF grammar under the start-symbol convention above.
- A nonempty word of length n has a binary CNF parse tree with exactly n terminal leaves.

Chomsky Normal Form (CNF)

Definition

A CFG is in Chomsky normal form when every production has one of the forms

$$A \rightarrow BC \quad \text{or} \quad A \rightarrow a,$$

where A, B, C are variables and a is a terminal. If $\varepsilon \in L(G)$, the additional rule $S_0 \rightarrow \varepsilon$ is allowed, provided the start variable S_0 never occurs on a right-hand side.

- Unit rules $A \rightarrow B$, mixed rules $A \rightarrow aB$, and right-hand sides of length greater than two are not in CNF.
- Every CFL has an equivalent CNF grammar under the start-symbol convention above.
- A nonempty word of length n has a binary CNF parse tree with exactly n terminal leaves.

A Reliable CNF Conversion Pipeline

Given $G = (V, T, S, P)$, apply the following language-preserving steps:

- 1 Introduce a fresh start variable S_0 with $S_0 \rightarrow S$.
- 2 Eliminate ε -productions, retaining $S_0 \rightarrow \varepsilon$ only when $\varepsilon \in L(G)$.
- 3 Eliminate unit productions such as $A \rightarrow B$.
- 4 Remove useless variables that are non-generating or unreachable.
- 5 In a right-hand side of length at least two, replace each terminal a by a variable A_a with $A_a \rightarrow a$.
- 6 Binarize every long rule. For example, $A \rightarrow BCD$ becomes $A \rightarrow BX$ and $X \rightarrow CD$.

Order matters

Removing ε - and unit productions can create new short rules, so perform those simplifications before terminal replacement and binarization.

A Reliable CNF Conversion Pipeline

Given $G = (V, T, S, P)$, apply the following language-preserving steps:

- 1 Introduce a fresh start variable S_0 with $S_0 \rightarrow S$.
- 2 Eliminate ε -productions, retaining $S_0 \rightarrow \varepsilon$ only when $\varepsilon \in L(G)$.
- 3 Eliminate unit productions such as $A \rightarrow B$.
- 4 Remove useless variables that are non-generating or unreachable.
- 5 In a right-hand side of length at least two, replace each terminal a by a variable A_a with $A_a \rightarrow a$.
- 6 Binarize every long rule. For example, $A \rightarrow BCD$ becomes $A \rightarrow BX$ and $X \rightarrow CD$.

Order matters

Removing ε - and unit productions can create new short rules, so perform those simplifications before terminal replacement and binarization.

Worked CNF Conversion

Convert the grammar

$$S \rightarrow aSb \mid ab,$$

which generates $\{a^n b^n : n \geq 1\}$.

- 1 Introduce variables for terminals used in longer productions:

$$A \rightarrow a, \quad B \rightarrow b.$$

- 2 Replace the terminal occurrences:

$$S \rightarrow ASB \mid AB.$$

- 3 Binarize the length-three rule using $X \rightarrow SB$:

$$S \rightarrow AX \mid AB, \quad X \rightarrow SB, \quad A \rightarrow a, \quad B \rightarrow b.$$

Every rule is now of the form variable-to-two-variables or variable-to-one-terminal.
For example, $S \Rightarrow AX \Rightarrow aX \Rightarrow aSB \Rightarrow aABB \Rightarrow aaBB \Rightarrow aabB \Rightarrow aabb$.

CYK Parsing: Dynamic-Programming Table

The Cocke–Younger–Kasami (CYK) algorithm decides membership for a grammar in CNF. For input $w = a_1 a_2 \cdots a_n$, define

$$T[i, \ell] = \{A \in V : A \Rightarrow^* a_i \cdots a_{i+\ell-1}\}.$$

Thus $T[i, \ell]$ contains the variables that generate the substring of length ℓ beginning at position i .

Recurrence

$$T[i, 1] = \{A : A \rightarrow a_i \in P\},$$

$$T[i, \ell] = \bigcup_{k=1}^{\ell-1} \{A : A \rightarrow BC, B \in T[i, k], C \in T[i+k, \ell-k]\}.$$

Fill the table from shorter substrings to longer substrings. The algorithm accepts exactly when

$$S \in T[1, n].$$

If the grammar permits $S \rightarrow \varepsilon$, test the empty input separately.

CYK Parsing: Dynamic-Programming Table

The Cocke–Younger–Kasami (CYK) algorithm decides membership for a grammar in CNF. For input $w = a_1 a_2 \cdots a_n$, define

$$T[i, \ell] = \{A \in V : A \Rightarrow^* a_i \cdots a_{i+\ell-1}\}.$$

Thus $T[i, \ell]$ contains the variables that generate the substring of length ℓ beginning at position i .

Recurrence

$$T[i, 1] = \{A : A \rightarrow a_i \in P\},$$

$$T[i, \ell] = \bigcup_{k=1}^{\ell-1} \{A : A \rightarrow BC, B \in T[i, k], C \in T[i+k, \ell-k]\}.$$

Fill the table from shorter substrings to longer substrings. The algorithm accepts exactly when

$$S \in T[1, n].$$

If the grammar permits $S \rightarrow \varepsilon$, test the empty input separately.

CYK Algorithm and Complexity

Bottom-up procedure

- 1 Initialize every length-one cell using terminal rules $A \rightarrow a_i$.
- 2 For lengths $\ell = 2, \dots, n$, examine every starting position i .
- 3 Try every split k of the substring into lengths k and $\ell - k$.
- 4 For each binary rule $A \rightarrow BC$, insert A when the left cell contains B and the right cell contains C .
- 5 Accept iff the start variable occurs in the top cell.

Let P_2 be the set of binary productions.

$$\text{Time } O(n^3 |P_2|), \quad \text{space } O(n^2 |V|).$$

For a fixed grammar, this is conventionally reported as $O(n^3)$ time and $O(n^2)$ table space. CNF conversion is preprocessing and its cost is separate from parsing a particular input word.

CYK Algorithm and Complexity

Bottom-up procedure

- 1 Initialize every length-one cell using terminal rules $A \rightarrow a_i$.
- 2 For lengths $\ell = 2, \dots, n$, examine every starting position i .
- 3 Try every split k of the substring into lengths k and $\ell - k$.
- 4 For each binary rule $A \rightarrow BC$, insert A when the left cell contains B and the right cell contains C .
- 5 Accept iff the start variable occurs in the top cell.

Let P_2 be the set of binary productions.

$$\text{Time } O(n^3 |P_2|), \quad \text{space } O(n^2 |V|).$$

For a fixed grammar, this is conventionally reported as $O(n^3)$ time and $O(n^2)$ table space. CNF conversion is preprocessing and its cost is separate from parsing a particular input word.

Worked CYK Example: Parsing *aabb*

Use the CNF grammar from the preceding conversion:

$$S \rightarrow AX \mid AB, \quad X \rightarrow SB, \quad A \rightarrow a, \quad B \rightarrow b.$$

Substring length	Start 1	Start 2	Start 3	Start 4
1	{A}	{A}	{B}	{B}
2	$\emptyset$	{S}	$\emptyset$	—
3	$\emptyset$	{X}	—	—
4	{S}	—	—	—

For example, $T[2, 2] = \{S\}$ because the substring *ab* splits into $\{A\}\{B\}$ and $S \rightarrow AB$. Then $T[2, 3] = \{X\}$ because $X \rightarrow SB$, and finally $T[1, 4] = \{S\}$ because $S \rightarrow AX$.

Decision

Since $S \in T[1, 4]$, CYK accepts *aabb*.

Important CKY/CYK Extensions

The names **CYK** and **CKY** denote the same core dynamic program; CKY is especially common in computational linguistics.

Extension	Significant distinction
Backpointer CKY	Stores successful split points and child variables, reconstructing a parse tree rather than answering membership only.
Packed-forest CKY	Shares common subtrees so that many parses of an ambiguous sentence can be represented compactly.
Viterbi CKY	For a probabilistic CFG, replaces Boolean values by maximum probabilities and returns the highest-probability parse.
Inside–outside CKY	Sums probabilities of all parses; the outside pass supports marginal probabilities and PCFG estimation.
Valiant's algorithm	Reformulates general CFG recognition using Boolean matrix multiplication, giving a theoretically subcubic alternative to the standard cubic recurrence.

Important CKY/CYK Extensions

The names **CYK** and **CKY** denote the same core dynamic program; CKY is especially common in computational linguistics.

Extension	Significant distinction
Backpointer CKY	Stores successful split points and child variables, reconstructing a parse tree rather than answering membership only.
Packed-forest CKY	Shares common subtrees so that many parses of an ambiguous sentence can be represented compactly.
Viterbi CKY	For a probabilistic CFG, replaces Boolean values by maximum probabilities and returns the highest-probability parse.
Inside–outside CKY	Sums probabilities of all parses; the outside pass supports marginal probabilities and PCFG estimation.
Valiant's algorithm	Reformulates general CFG recognition using Boolean matrix multiplication, giving a theoretically subcubic alternative to the standard cubic recurrence.

CYK in Practice and Important Alternatives

Real-world connection. In natural-language processing, probabilistic CKY ranks competing constituency analyses using a PCFG, while a packed chart shares repeated subtrees.

Method	Grammar requirement	Main distinction
CYK/CKY	CNF or binarized CFG	Simple span dynamic programming; cubic worst-case time.
Earley parser	Arbitrary CFG	Prediction, scanning, and completion; no CNF conversion and cubic worst case.
GLR/Tomita	Generalized LR treatment	Uses a graph-structured stack to pursue conflicts and represent ambiguous parses.
LL/LR parsers	Restricted deterministic CFGs	Typically linear-time parsing; dominant for programming-language syntax.

Terminology: Earley, GLR, and LL/LR are alternatives, not variants of the CYK recurrence; their grammar requirements and engineering goals differ. Blue method names link to the core paper or a standard resource.

Greibach Normal Form (GNF)

Definition

A CFG is in Greibach normal form when every production has the form

$$A \rightarrow aA_1A_2 \cdots A_k, \quad k \geq 0,$$

where a is a terminal and each A_i is a variable. As in CNF, a fresh start rule $S_0 \rightarrow \varepsilon$ may be allowed when the language contains ε .

- Every non- ε production begins with exactly one terminal.
- Each step of a leftmost derivation produces exactly one new terminal; hence a word of length n is derived in exactly n steps.
- Every CFL has an equivalent GNF grammar, subject to the usual treatment of ε .

Greibach Normal Form (GNF)

Definition

A CFG is in Greibach normal form when every production has the form

$$A \rightarrow aA_1A_2 \cdots A_k, \quad k \geq 0,$$

where a is a terminal and each A_i is a variable. As in CNF, a fresh start rule $S_0 \rightarrow \varepsilon$ may be allowed when the language contains ε .

- Every non- ε production begins with exactly one terminal.
- Each step of a leftmost derivation produces exactly one new terminal; hence a word of length n is derived in exactly n steps.
- Every CFL has an equivalent GNF grammar, subject to the usual treatment of ε .

A Systematic GNF Conversion Outline

A full GNF construction is more involved than CNF conversion. A standard outline is:

- 1 Introduce a fresh start variable and remove null, unit, and useless productions.
- 2 Replace terminals occurring after the first position by helper variables, such as $B_b \rightarrow b$.
- 3 Order the variables $A_1, \dots, A_n$.
- 4 Whenever a rule for A_i begins with A_j for $j < i$, substitute the productions of A_j . Eliminate any immediate left recursion thereby exposed.
- 5 Process variables in reverse order, repeatedly substituting a leading variable until every right-hand side begins with a terminal.
- 6 Put any helper variables introduced during left-recursion removal into the same terminal-leading form.

Practical perspective

The construction proves expressive equivalence but may increase grammar size substantially. Production parsers usually target LL/LR-style grammars directly rather than converting a programming-language grammar to GNF.

Worked GNF Conversion: Removing Left Recursion

The grammar

$$S \rightarrow Sa \mid b$$

generates ba^* , but is not in GNF: one rule begins with a variable and it is immediately left-recursive.

Equivalent GNF grammar

$$S \rightarrow b \mid bA, \quad A \rightarrow a \mid aA.$$

Every production begins with a terminal followed only by variables. The derivation

$$S \Rightarrow bA \Rightarrow baA \Rightarrow baa$$

generates baa . In general, $S \rightarrow b$ produces zero a 's, while $S \rightarrow bA$ followed by the rules for A produces one or more a 's.

Typical GNF strategy

Order variables, substitute productions that begin with earlier variables, eliminate left recursion, and continue until every rule begins with a terminal.

Do Normal Forms Remove Ambiguity?

No

CNF and GNF constrain production shape; they do not guarantee a unique parse tree.

Ambiguous CNF grammar

$$S \rightarrow SS \mid a.$$

The word aaa has the groupings $(aa)a$ and $a(aa)$, so a grammar can be both CNF and ambiguous.

Ambiguous GNF grammar

$$S \rightarrow aS \mid aA \mid a, \quad A \rightarrow a.$$

The word aa has derivations $S \Rightarrow aS \Rightarrow aa$ and $S \Rightarrow aA \Rightarrow aa$.

- If a language is inherently ambiguous, every equivalent normal-form grammar remains ambiguous.
- If only the original grammar is ambiguous, ambiguity must be handled by grammar redesign or explicit disambiguation rules—not merely by normal-form conversion.

Do Normal Forms Remove Ambiguity?

No

CNF and GNF constrain production shape; they do not guarantee a unique parse tree.

Ambiguous CNF grammar

$$S \rightarrow SS \mid a.$$

The word aaa has the groupings $(aa)a$ and $a(aa)$, so a grammar can be both CNF and ambiguous.

Ambiguous GNF grammar

$$S \rightarrow aS \mid aA \mid a, \quad A \rightarrow a.$$

The word aa has derivations
 $S \Rightarrow aS \Rightarrow aa$ and $S \Rightarrow aA \Rightarrow aa$.

- If a language is inherently ambiguous, every equivalent normal-form grammar remains ambiguous.
- If only the original grammar is ambiguous, ambiguity must be handled by grammar redesign or explicit disambiguation rules—not merely by normal-form conversion.

GNF Versus an s-Grammar

Form	Restriction
GNF	Every production is $A \rightarrow a\alpha$, where $a \in T$ and $\alpha \in V^*$.
s-grammar	It is in GNF, and for each pair (A, a) there is at most one production whose left side is A and whose first terminal is a .

s-grammar

$$S \rightarrow aA \mid bB,$$

$$A \rightarrow aA \mid b,$$

$$B \rightarrow bB \mid a.$$

The next terminal uniquely selects a rule for the current variable.

GNF, but not an s-grammar

$$S \rightarrow aA \mid aB,$$

$$A \rightarrow b, \quad B \rightarrow c.$$

Both productions for S begin with a , violating uniqueness.

Other Useful Grammar Forms and Restrictions

CNF and GNF are the principal universal normal forms for CFGs, but related forms are useful in specialized settings.

Form or restriction	Main purpose and limitation
Reduced/proper grammar	Removes useless, null, and unit productions; commonly used as preprocessing.
Binary or 2-normal form	Bounds right-hand-side length by two but may allow mixtures such as $A \rightarrow aB$; weaker than CNF.
Operator form	Avoids adjacent variables and supports operator-precedence techniques for expression-like grammars; not a universal deterministic parsing solution.
$LL(k)$, $LR(k)$ restrictions	Enable efficient deterministic parsing, often linear in input length, but not every CFL has such a grammar.
Kuroda normal form	A normal form for context-sensitive grammars, not a CFG normal form.

These forms answer different questions: descriptive equivalence, proof convenience, or efficient deterministic parsing.

Closure Properties of CFLs

Context-free languages are closed under several constructive operations. If L_1 and L_2 are CFLs, then the following are CFLs:

- $L_1 \cup L_2$ (union), $L_1 L_2$ (concatenation), and L_1^* (Kleene star);
- L_1^R (reversal);
- $h(L_1)$ for a homomorphism h ;
- $h^{-1}(L_1)$ for an inverse homomorphism; and
- $L_1 \cap R$ and $L_1 \setminus R$ when R is regular.

Not all regular-language properties carry over

CFLs are not closed under intersection, complement, or difference of two arbitrary CFLs.

Closure Properties of CFLs

Context-free languages are closed under several constructive operations. If L_1 and L_2 are CFLs, then the following are CFLs:

- $L_1 \cup L_2$ (union), $L_1 L_2$ (concatenation), and L_1^* (Kleene star);
- L_1^R (reversal);
- $h(L_1)$ for a homomorphism h ;
- $h^{-1}(L_1)$ for an inverse homomorphism; and
- $L_1 \cap R$ and $L_1 \setminus R$ when R is regular.

Not all regular-language properties carry over

CFLs are not closed under intersection, complement, or difference of two arbitrary CFLs.

Closure by Grammar Construction

Let G_1 and G_2 have disjoint variable sets and start variables S_1, S_2 . Introduce a fresh start variable S .

Language	New productions	Why it works
$L_1 \cup L_2$	$S \rightarrow S_1 \mid S_2$	choose either grammar
$L_1 L_2$	$S \rightarrow S_1 S_2$	generate one word from each
L_1^*	$S \rightarrow S_1 S \mid \varepsilon$	generate zero or more factors

Reversal

Reverse the right-hand side of every production. Reversing each local child order reverses the yield of every parse tree, so the new grammar generates L_1^R .

Worked Closure Examples

Let

$$L_1 = \{a^n b^n : n \geq 0\}, \quad L_2 = \{a^n c^n : n \geq 0\}.$$

Grammars $A \rightarrow aAb \mid \varepsilon$ and $B \rightarrow aBc \mid \varepsilon$ generate L_1 and L_2 .

Union and concatenation

$$S_U \rightarrow A \mid B$$

generates $L_1 \cup L_2$,

$$S_C \rightarrow AB$$

generates $L_1 L_2 = \{a^i b^i a^j c^j : i, j \geq 0\}$.

Homomorphic image

If $h(a) = 01$ and $h(b) = 2$, then

$$h(L_1) = \{(01)^n 2^n : n \geq 0\},$$

generated directly by $H \rightarrow 01H2 \mid \varepsilon$.

Intersection with a Regular Language

Theorem

If L is context-free and R is regular, then $L \cap R$ is context-free.

Proof sketch. Let an NPDA P recognize L and a DFA D recognize R . Construct a product PDA whose finite-control states are pairs (p, q) :

- the first component follows P and retains its stack;
- the second component follows D whenever an input symbol is consumed;
- on an ε -move of P , the DFA component remains unchanged; and
- accept only when both components accept.

The product still has exactly one stack, so it is a PDA.

Useful corollary

If R is regular, then $L \setminus R = L \cap \overline{R}$ is context-free.

Intersection with a Regular Language

Theorem

If L is context-free and R is regular, then $L \cap R$ is context-free.

Proof sketch. Let an NPDA P recognize L and a DFA D recognize R . Construct a product PDA whose finite-control states are pairs (p, q) :

- the first component follows P and retains its stack;
- the second component follows D whenever an input symbol is consumed;
- on an ε -move of P , the DFA component remains unchanged; and
- accept only when both components accept.

The product still has exactly one stack, so it is a PDA.

Useful corollary

If R is regular, then $L \setminus R = L \cap \overline{R}$ is context-free.

Worked Example: Filtering a CFL with a Regular Language

Let P be the language of binary palindromes and let

$$R = 0^*10^*,$$

the regular language of strings containing exactly one 1. A string in both languages must have equally many 0's on the two sides of its unique 1. Therefore

$$P \cap R = \{0^n10^n : n \geq 0\}.$$

This intersection is context-free, with grammar

$$S \rightarrow 0S0 \mid 1.$$

What the theorem does and does not say

Intersecting a CFL with a regular language is always safe. Intersecting two CFLs may still happen to produce a CFL, but the class has no closure guarantee for that operation.

Nonclosure Under Intersection

Consider

$$L_1 = \{a^n b^n c^m \mid n, m \geq 0\},$$

$$L_2 = \{a^m b^n c^n \mid n, m \geq 0\}.$$

Both are context-free: L_1 matches the a and b blocks, while L_2 matches the b and c blocks. However,

$$L_1 \cap L_2 = \{a^n b^n c^n \mid n \geq 0\},$$

which is not context-free.

Consequences

CFLs are not closed under intersection. They are also not closed under complement: otherwise De Morgan's law and union closure would imply intersection closure. Since $\bar{L} = \Sigma^* \setminus L$, CFLs are not closed under arbitrary difference either.

Nonclosure Under Intersection

Consider

$$L_1 = \{a^n b^n c^m \mid n, m \geq 0\},$$

$$L_2 = \{a^m b^n c^n \mid n, m \geq 0\}.$$

Both are context-free: L_1 matches the a and b blocks, while L_2 matches the b and c blocks. However,

$$L_1 \cap L_2 = \{a^n b^n c^n \mid n \geq 0\},$$

which is not context-free.

Consequences

CFLs are not closed under intersection. They are also not closed under complement: otherwise De Morgan's law and union closure would imply intersection closure. Since $\bar{L} = \Sigma^* \setminus L$, CFLs are not closed under arbitrary difference either.

Decision Properties of CFGs

Problem	Decidable?	Main idea
Membership: $w \in L(G)$	Yes	parsing; CYK gives a polynomial algorithm after CNF conversion
Emptiness: $L(G) = \emptyset$	Yes	mark variables that can derive terminal strings
Finiteness of $L(G)$	Yes	analyze useful productive recursion
Universality: $L(G) = \Sigma^*$	No	computation-history reduction
Equivalence: $L(G_1) = L(G_2)$	No	universality reduces to equivalence
Inclusion: $L(G_1) \subseteq L(G_2)$	No	universality reduces to inclusion
Grammar ambiguity	No	PCP reduction
Intersection emptiness	No	PCP reduction

Decidable membership is a major contrast with general Turing-machine acceptance.

Parse-Tree Intuition for Pumping

A CFG has finitely many variables. In a sufficiently tall parse tree, some variable A must repeat along a root-to-leaf path:

$$S \Rightarrow^* uAz, \quad A \Rightarrow^* vAy, \quad A \Rightarrow^* x.$$

Combining these derivations gives

$$S \Rightarrow^* uvxyz.$$

The repeated A -subtree may be omitted or repeated:

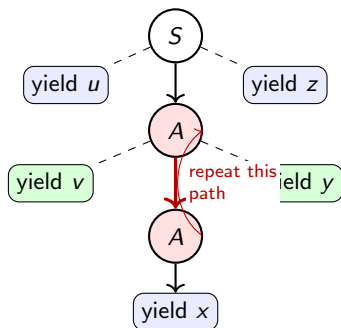
$$S \Rightarrow^* uv^i xy^i z \quad (i \geq 0).$$

Why two substrings pump

Repeating the derivation from the outer A to the inner A duplicates material on both sides of the inner subtree, producing the two pumpable pieces v and y .

After conversion to Chomsky normal form with k variables, one may take a bound such as $p = 2^k$. A binary parse tree with at least p leaves has a sufficiently long path to repeat a variable, while choosing the repetition near the leaves ensures $|vxy| \leq p$.

Visualizing the Repeating Part of the Parse Tree



The repeated variable creates a derivation loop:

$$A \Rightarrow^* vAy.$$

$$i = 0 : \quad uxz,$$

$$i = 1 : \quad uvxyz,$$

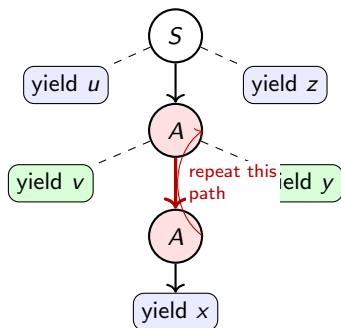
$$i = 2 : \quad uvvxyyz,$$

$$i = 3 : \quad uvvvxyyyz.$$

Analogy with regular pumping

A repeated DFA state pumps one loop substring. A repeated CFG variable pumps the material produced on *both sides* of the nested copy: v and y .

Visualizing the Repeating Part of the Parse Tree



The repeated variable creates a derivation loop:

$$A \Rightarrow^* vAy.$$

$$i = 0 : \quad uxz,$$

$$i = 1 : \quad uvxyz,$$

$$i = 2 : \quad uvvxyyz,$$

$$i = 3 : \quad uvvvxyyyz.$$

Analogy with regular pumping

A repeated DFA state pumps one loop substring. A repeated CFG variable pumps the material produced on *both sides* of the nested copy: v and y .

Formal CFL Pumping Lemma

Theorem

If L is context-free, then there exists a pumping length $p \geq 1$ such that every $s \in L$ with $|s| \geq p$ can be written

$$s = uvxyz$$

so that:

- 1 $|vxy| \leq p$;
- 2 $|vy| > 0$; and
- 3 $uv^i xy^i z \in L$ for every $i \geq 0$.

Compare with the regular pumping lemma

The CFL lemma pumps two pieces, v and y , using the same exponent. The bounded window vxy may occur anywhere in s ; it is not forced into the first p symbols.

Formal CFL Pumping Lemma

Theorem

If L is context-free, then there exists a pumping length $p \geq 1$ such that every $s \in L$ with $|s| \geq p$ can be written

$$s = uvxyz$$

so that:

- 1 $|vxy| \leq p$;
- 2 $|vy| > 0$; and
- 3 $uv^i xy^i z \in L$ for every $i \geq 0$.

Compare with the regular pumping lemma

The CFL lemma pumps two pieces, v and y , using the same exponent. The bounded window vxy may occur anywhere in s ; it is not forced into the first p symbols.

The CFL Pumping Game

To prove that L is not context-free, argue by contradiction:

- 1 **Opponent chooses** an arbitrary pumping length p .
- 2 **You choose** a strategic $s \in L$ with $|s| \geq p$.
- 3 **Opponent chooses** any split $s = uvxyz$ satisfying $|vxy| \leq p$ and $|vy| > 0$.
- 4 **You choose**, after seeing the split, some $i \geq 0$ for which $uv^i xy^i z \notin L$.

Quantifier discipline

Your witness string must defeat *every* legal decomposition. Choosing one convenient split and breaking it proves nothing.

The CFL Pumping Game

To prove that L is not context-free, argue by contradiction:

- 1 **Opponent chooses** an arbitrary pumping length p .
- 2 **You choose** a strategic $s \in L$ with $|s| \geq p$.
- 3 **Opponent chooses** any split $s = uvxyz$ satisfying $|vxy| \leq p$ and $|vy| > 0$.
- 4 **You choose**, after seeing the split, some $i \geq 0$ for which $uv^i xy^i z \notin L$.

Quantifier discipline

Your witness string must defeat *every* legal decomposition. Choosing one convenient split and breaking it proves nothing.

The CFL Pumping Game

To prove that L is not context-free, argue by contradiction:

- 1 **Opponent chooses** an arbitrary pumping length p .
- 2 **You choose** a strategic $s \in L$ with $|s| \geq p$.
- 3 **Opponent chooses** any split $s = uvxyz$ satisfying $|vxy| \leq p$ and $|vy| > 0$.
- 4 **You choose**, after seeing the split, some $i \geq 0$ for which $uv^i xy^i z \notin L$.

Quantifier discipline

Your witness string must defeat *every* legal decomposition. Choosing one convenient split and breaking it proves nothing.

Game Comparison: Regular Versus Context-Free

Regular pumping lemma	CFL pumping lemma
Split $s = xyz$	Split $s = uvxyz$
Pump one substring: xy^iz	Pump two substrings: uv^ixy^iz
$ xy \leq p$ forces y near the left end	$ vxy \leq p$ gives a floating bounded window
$ y > 0$	$ vy > 0$; either v or y may be empty
Opponent chooses the legal split	Opponent chooses the legal split
You choose i after seeing the split	You choose i after seeing the split

Both lemmas are necessary conditions, primarily useful for proving nonmembership in a language class.

Example 1: $\{a^n b^n c^n \mid n \geq 0\}$ Is Not Context-Free

Assume the language is context-free and let p be its pumping length. Choose

$$s = a^p b^p c^p.$$

For any legal $s = uvxyz$, the window vxy has length at most p , so it cannot contain both an a and a c : the entire block b^p lies between them. Thus v and y affect symbols from at most two of the three blocks.

Pump down with $i = 0$. Since $|vy| > 0$, at least one block loses a symbol, while at least one other block remains of length p . Deletion preserves the order $a^* b^* c^*$, but the three counts cannot all remain equal. Therefore

$$uv^0 xy^0 z = uxz \notin L,$$

contradicting the pumping lemma. Hence the language is not context-free.

Example 1: $\{a^n b^n c^n \mid n \geq 0\}$ Is Not Context-Free

Assume the language is context-free and let p be its pumping length. Choose

$$s = a^p b^p c^p.$$

For any legal $s = uvxyz$, the window vxy has length at most p , so it cannot contain both an a and a c : the entire block b^p lies between them. Thus v and y affect symbols from at most two of the three blocks.

Pump down with $i = 0$. Since $|vy| > 0$, at least one block loses a symbol, while at least one other block remains of length p . Deletion preserves the order $a^* b^* c^*$, but the three counts cannot all remain equal. Therefore

$$uv^0 xy^0 z = uxz \notin L,$$

contradicting the pumping lemma. Hence the language is not context-free.

Example 2: Unary Perfect Squares Are Not Context-Free

Let

$$L = \{a^{n^2} \mid n \geq 0\}.$$

Assume L is context-free with pumping length p , and choose $s = a^{p^2}$. For any legal decomposition $s = uvxyz$, let

$$d = |v| + |y|.$$

The pumping conditions give $1 \leq d \leq p$. Pumping up with $i = 2$ produces a string of length

$$p^2 + d.$$

But

$$p^2 < p^2 + d \leq p^2 + p < (p + 1)^2.$$

Its length lies strictly between consecutive perfect squares, so $uv^2xy^2z \notin L$. Therefore L is not context-free.

Example 3: A Coupled Three-Block Language

Let

$$L = \{a^n b^n c^{2n} : n \geq 0\}.$$

Assume L is context-free with pumping length p , and choose $s = a^p b^p c^{2p}$. Any window vxy of length at most p touches at most two adjacent blocks, so it cannot alter all three counts.

Pump down with $i = 0$. At least one symbol is deleted because $|vy| > 0$.

- If an a or b is deleted without equal changes to both, then $|a| = |b|$ fails.
- If the a - and b -counts both decrease equally, the untouched c -count is no longer twice the new a -count.
- If only the b/c region is affected, either $|a| = |b|$ fails or, when no b is deleted, $|c| = 2|a|$ fails.

Thus every legal split can be defeated, so L is not context-free.

Example 4: Quadratically Related Blocks

Let

$$L = \{a^n b^{n^2} : n \geq 0\}.$$

Assume L is context-free with pumping length p . Set $N = p + 1$ and choose $s = a^N b^{N^2}$. For an arbitrary legal split, let

$$\alpha = |vy|_a, \quad \beta = |vy|_b.$$

Then $1 \leq \alpha + \beta \leq p$. Pump down. If the resulting string remained in L , its counts would have to satisfy

$$N^2 - \beta = (N - \alpha)^2, \quad \text{so} \quad \beta = 2N\alpha - \alpha^2.$$

If $\alpha = 0$, the equation requires $\beta = 0$, contradicting $|vy| > 0$. If $\alpha \geq 1$, then $\alpha \leq p = N - 1$ and

$$2N\alpha - \alpha^2 = \alpha(2N - \alpha) \geq 2N - 1 > p \geq \beta,$$

again impossible. Hence L is not context-free.

Example 5: The Copy Language $\{ww\}$

Let $L = \{ww : w \in \{a, b\}^*\}$. Assume L is context-free with pumping length p , and choose

$$s = a^p b^p a^p b^p = (a^p b^p)(a^p b^p).$$

A window vxy of length at most p lies in one block or crosses one boundary; it cannot reach two corresponding blocks.

- Pump down. If all four runs remain nonempty, at most two adjacent run lengths change. But any square in $a^+ b^+ a^+ b^+$ must have its first and third run lengths equal and its second and fourth run lengths equal. At least one equality is destroyed.
- If pumping down would delete an entire length- p run, then $|vy| = p$, $x = \varepsilon$, and the pumpable material is exactly that run. Pump up instead; precisely one run doubles, again destroying the required pair of equal run lengths.

Thus every legal decomposition has a bad pumping exponent, so L is not context-free.

Example 6: Two Crossed Dependencies

Let

$$L = \{a^n b^m c^n d^m : n, m \geq 1\}.$$

Membership requires the distant pairs $|a| = |c|$ and $|b| = |d|$. Assume L is context-free with pumping length p , and choose

$$s = a^p b^p c^p d^p.$$

Since $|vxy| \leq p$, the pumpable window touches at most two adjacent blocks. It therefore cannot touch both a and c , nor both b and d . Pump down with $i = 0$. Because $|vy| > 0$, at least one block count decreases, while its distant dependent partner remains p . Consequently either $|a| \neq |c|$ or $|b| \neq |d|$, so

$$uxz \notin L.$$

This contradiction proves that L is not context-free.

Example 7: Strictly Increasing Block Counts

Let

$$L = \{a^i b^j c^k : i < j < k\}.$$

Assume L is context-free with pumping length p , and choose the deliberately tight gaps

$$s = a^p b^{p+1} c^{p+2}.$$

For an arbitrary split, let $\Delta_a, \Delta_b, \Delta_c$ be the numbers of a 's, b 's, and c 's in vy . The bounded window cannot touch all three blocks, and $|vy| > 0$, so these three changes are not all equal.

- Pumping up preserves both inequalities only if $\Delta_a \leq \Delta_b \leq \Delta_c$.
- Pumping down preserves both inequalities only if $\Delta_a \geq \Delta_b \geq \Delta_c$.

Both conditions can hold only when $\Delta_a = \Delta_b = \Delta_c$, which is impossible here. Hence either $i = 2$ or $i = 0$ violates a strict inequality, proving L non-context-free.

Pumping-Lemma Pitfalls and Limitations

- Do not choose the decomposition; the proof must cover all legal u, v, x, y, z .
- Do not assume both v and y are nonempty; only $|vy| > 0$ is guaranteed.
- Pump v and y with the same exponent i .
- The window vxy may cross one or more symbol boundaries; justify every case.
- Choose i after the opponent reveals the split. Often $i = 0$ is cleaner than $i = 2$.
- The lemma is necessary, not sufficient: satisfying it does not prove that a language is context-free.

When the lemma is inconclusive

Try closure properties, intersection with a regular language, Ogden's lemma, or a direct structural argument.

Why Non-CFL Proofs Matter in Software Systems

A context-free parser has one stack-like source of unbounded memory. Some useful program constraints require comparisons that are not naturally context-free.

Constraint	Typical implementation
Proper nesting of parentheses and blocks	CFG parser or pushdown automaton
A declared identifier must be used consistently	Symbol table and semantic analysis
Function call arguments must match a declaration	Type checking after parsing
Two arbitrarily long regions must be exact copies	Explicit comparison, hashes, or a later validation pass
Several distant counts must agree simultaneously	Attributes, counters, or general program logic

Engineering lesson

Compiler front ends deliberately separate context-free syntax from context-sensitive semantic checks. A non-CFL proof explains why merely adding more CFG productions cannot enforce every desired constraint.

Backus–Naur Form is a notation for CFG productions:

$$\begin{aligned}\langle \text{assignment} \rangle &::= \langle \text{id} \rangle = \langle \text{expression} \rangle \\ \langle \text{id} \rangle &::= x \mid y \mid z.\end{aligned}$$

Extended BNF adds notation for repetition and optional material. For example,

$$\langle \text{list} \rangle ::= \langle \text{id} \rangle \{ ", " \langle \text{id} \rangle \}$$

can be expanded into ordinary CFG productions:

$$L \rightarrow \text{id } R, \quad R \rightarrow, \text{id } R \mid \epsilon.$$

EBNF conventions vary between standards, so braces, brackets, and parentheses must be defined.

Backus–Naur Form is a notation for CFG productions:

$$\begin{aligned}\langle \text{assignment} \rangle &::= \langle \text{id} \rangle = \langle \text{expression} \rangle \\ \langle \text{id} \rangle &::= x \mid y \mid z.\end{aligned}$$

Extended BNF adds notation for repetition and optional material. For example,

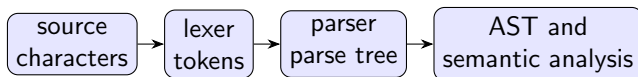
$$\langle \text{list} \rangle ::= \langle \text{id} \rangle \{ ", " \langle \text{id} \rangle \}$$

can be expanded into ordinary CFG productions:

$$L \rightarrow \text{id } R, \quad R \rightarrow, \text{id } R \mid \varepsilon.$$

EBNF conventions vary between standards, so braces, brackets, and parentheses must be defined.

CFGs in a Compiler Pipeline

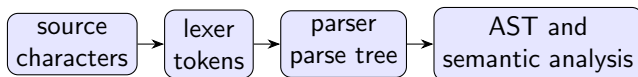


- Lexical rules are usually regular and identify tokens such as identifiers and numbers.
- A CFG organizes tokens into nested syntactic structure.
- Semantic analysis checks properties not conveniently captured by a CFG, such as types, declarations, and scope.

Syntax is not semantics

A program can be grammatically valid but still contain an undeclared variable or type error.

CFGs in a Compiler Pipeline



- Lexical rules are usually regular and identify tokens such as identifiers and numbers.
- A CFG organizes tokens into nested syntactic structure.
- Semantic analysis checks properties not conveniently captured by a CFG, such as types, declarations, and scope.

Syntax is not semantics

A program can be grammatically valid but still contain an undeclared variable or type error.

Top-Down and Bottom-Up Views

A derivation begins with S and expands variables until the input string appears. This is a **top-down** view of syntax.

- A top-down parser predicts which production should expand the current variable.
- A bottom-up parser starts from the input and repeatedly recognizes substrings corresponding to production right-hand sides, eventually reducing to S .

Connection to the next unit

Pushdown automata provide the machine model for context-free languages. Their stack records unfinished grammatical structure during recognition or parsing.

Concept Checks

- 1 Is S always a sentential form? Is it always a sentence?
- 2 Why do two arbitrary derivations of the same string not necessarily prove ambiguity?
- 3 Which labels may occur at the leaves of a partial derivation tree but not a completed derivation tree?
- 4 Does an ambiguous grammar necessarily generate an inherently ambiguous language?
- 5 Does conversion to CNF or GNF guarantee an unambiguous grammar?
- 6 How does Viterbi CKY differ from Boolean membership CYK?
- 7 Which part of the expression grammar enforces multiplication precedence?
- 8 Why can a CFG describe syntax but not all static semantic rules of a programming language?

Construction Exercises

- 1 Give a leftmost derivation of 000111 using $S \rightarrow 0S1 \mid \varepsilon$.
- 2 Draw a partial derivation tree with frontier 0S0, then complete it to a tree for 0110 using $S \rightarrow 0S0 \mid 1S1 \mid 0 \mid 1 \mid \varepsilon$.
- 3 Design a CFG for $\{0^i 1^j \mid i, j \geq 0\}$ and explain why it is correct.
- 4 For $S \rightarrow SS \mid a$, give two leftmost derivations of aaa .
- 5 Modify the unambiguous expression grammar so exponentiation $\uparrow$ has higher precedence than multiplication and is right-associative.
- 6 Prove, using two inclusions, that $S \rightarrow aSb \mid \varepsilon$ generates $\{a^n b^n \mid n \geq 0\}$.

Exercises: Simplifying CFGs

① In

$$S \rightarrow ABC \mid d, \quad A \rightarrow a \mid \varepsilon, \quad B \rightarrow bB \mid \varepsilon, \quad C \rightarrow c,$$

find all nullable variables and list the rules obtained from $S \rightarrow ABC$ when null productions are removed.

② Eliminate unit productions from

$$S \rightarrow A \mid d, \quad A \rightarrow B \mid a, \quad B \rightarrow bB \mid c.$$

③ Identify the useless variables in

$$S \rightarrow AB \mid a, \quad A \rightarrow a, \quad B \rightarrow b, \quad C \rightarrow cC \mid c, \quad D \rightarrow dD.$$

④ Explain why the generating-symbol pass is normally performed before the reachability pass.

Exercises: CNF, GNF, and s-Grammars

- ❶ Which of the rules below are permitted in CNF? Explain each answer.

$$A \rightarrow BC, \quad A \rightarrow a, \quad A \rightarrow aB, \quad A \rightarrow B, \quad A \rightarrow BCD.$$

- ❷ Convert $S \rightarrow aSb \mid \varepsilon$ to CNF, using the standard fresh start-symbol convention to preserve ε .
- ❸ Convert the grammar

$$S \rightarrow AB \mid b, \quad A \rightarrow aA \mid a, \quad B \rightarrow bB \mid b$$

to GNF by substituting the productions for A into $S \rightarrow AB$.

- ❹ Explain why $S \rightarrow aA \mid aB, A \rightarrow b, B \rightarrow c$ is in GNF but is not an s-grammar.
- ❺ Prove that a GNF leftmost derivation of a terminal word of length n uses exactly n production steps.
- ❻ Run CYK on $abab$ using $S \rightarrow AX \mid AB, X \rightarrow SB, A \rightarrow a$, and $B \rightarrow b$. Does the grammar generate the string?

Exercises: CFL Properties

- 1 For a CFL L and regular language R , which are guaranteed to be context-free: $L \cup R$, $L \cap R$, $L \setminus R$, and $R \setminus L$?
- 2 Let $L_1 = L(G_1)$ and $L_2 = L(G_2)$. Give grammar constructions for $L_1 \cup L_2$, $L_1 L_2$, and L_1^* .
- 3 Construct a CFG for $\{a^i b^j c^k \mid i = j \text{ or } j = k\}$ using union closure.
- 4 Explain why closure under complement, together with union closure, would imply closure under intersection.
- 5 Classify each CFG question as decidable or undecidable: membership, emptiness, finiteness, universality, equivalence, and ambiguity.
- 6 Why does nonclosure under intersection not mean that the intersection of every two CFLs is non-context-free?

Exercises: CFL Pumping Lemma

- 1 Use the CFL pumping lemma to prove $\{0^n 1^n 0^n \mid n \geq 0\}$ is not context-free.
- 2 Prove that $\{a^{n^3} \mid n \geq 0\}$ is not context-free.
- 3 A student chooses $s = a^p b^p c^p$ and writes “let $v = a$ and $y = b$.” Explain the quantifier error.
- 4 In a valid split $s = uvxyz$, may v be empty? May y be empty? May both be empty?
- 5 Why does finding one decomposition that pumps successfully not prove that a language is context-free?
- 6 For $s = a^p b^p c^p$, explain precisely why a window of length at most p cannot touch all three blocks.

Common Mistakes to Avoid

- Giving examples instead of proving both language inclusions.
- Leaving a variable as a parse-tree leaf in a completed parse.
- Confusing ε with the empty language $\emptyset$.
- Claiming ambiguity from two derivations that differ only in expansion order.
- Forgetting base productions, causing every derivation to continue forever.
- Adding precedence rules without checking associativity.
- Calling a grammar CNF before eliminating unit, mixed-terminal, or length-greater-than-two productions.
- Confusing GNF with an s -grammar; GNF alone does not require a unique rule for each leading terminal.
- Assuming that removing ambiguity from one grammar proves the language is never inherently ambiguous.

Summary

- CFGs use recursive productions $A \rightarrow \alpha$ to generate context-free languages.
- Derivations describe substitution sequences; parse trees capture hierarchical structure.
- Grammar correctness is normally proved with soundness and completeness inclusions.
- Ambiguity is structural: one string has two parse trees or two leftmost derivations.
- CNF gives binary productions for proofs and CYK parsing; for a fixed grammar, CYK uses $O(n^3)$ time and $O(n^2)$ table space.
- GNF makes each production begin with a terminal, while an s -grammar adds a uniqueness condition on the leading terminal.
- CFLs are closed under union, concatenation, star, and intersection with regular languages, but not under arbitrary intersection or complement.
- The CFL pumping lemma pumps two bounded pieces and is used to prove languages non-context-free.
- Carefully designed variables encode precedence, associativity, nesting, and statement structure.
- CFG membership is decidable, while equivalence, universality, and ambiguity are undecidable.

Appendix Road Map

- Deeper motivation: language classes, compiler phases, and the limits of automatic program analysis;
- Short answers and solution blueprints for the exercises;
- advanced theorems about deterministic CFLs, semilinearity, strengthened pumping, and structural representations;
- surprising examples that separate closely related language classes;
- fully worked, newly phrased book-style challenge problems;
- advanced or enrichment slides that may be omitted in a shorter classroom delivery;
- guidance for selecting a compact subset of examples and supplementary references.

Presentation use

Stop at the preceding Summary slide for the core lecture. Continue into this appendix only when time permits or when reviewing exercises.

The Chomsky Hierarchy: What Memory Makes Possible

Language classes classify *sets of strings*, not entire application areas. More powerful models can enforce more kinds of constraints.

Language class	Equivalent recognizer	Memory available
Type 3: regular	Finite automaton	Finitely many states; no auxiliary stack
Type 2: context-free	Nondeterministic PDA	Finite control and one unbounded stack
Type 1: context-sensitive	Nondeterministic linear bounded automaton (LBA)	Read/write workspace bounded by $O(n)$ cells for input length n
Type 0: recursively enumerable	Turing machine	Unbounded read/write tape; each finite computation uses only finitely many cells

$$\text{REG} \subsetneq \text{CFL} \subsetneq \text{CSL} \subsetneq \text{Decidable} \subsetneq \text{Recognizable}.$$

Recursively enumerable means *Turing-recognizable*, not necessarily decidable. The decidable class is a useful extra distinction, not a fifth numbered grammar type.

Empty-word convention: allow the isolated start rule $S_0 \rightarrow \epsilon$ when needed so the displayed inclusions include languages containing ϵ . See [UT Austin notes: language hierarchy and automata](#).

The Chomsky Hierarchy: What Memory Makes Possible

Language classes classify *sets of strings*, not entire application areas. More powerful models can enforce more kinds of constraints.

Language class	Equivalent recognizer	Memory available
Type 3: regular	Finite automaton	Finitely many states; no auxiliary stack
Type 2: context-free	Nondeterministic PDA	Finite control and one unbounded stack
Type 1: context-sensitive	Nondeterministic linear bounded automaton (LBA)	Read/write workspace bounded by $O(n)$ cells for input length n
Type 0: recursively enumerable	Turing machine	Unbounded read/write tape; each finite computation uses only finitely many cells

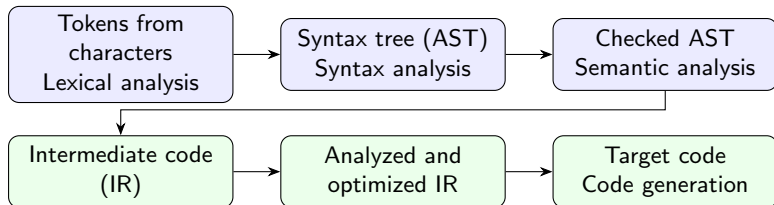
$$\text{REG} \subsetneq \text{CFL} \subsetneq \text{CSL} \subsetneq \text{Decidable} \subsetneq \text{Recognizable}.$$

Recursively enumerable means *Turing-recognizable*, not necessarily decidable. The decidable class is a useful extra distinction, not a fifth numbered grammar type.

Empty-word convention: allow the isolated start rule $S_0 \rightarrow \varepsilon$ when needed so the displayed inclusions include languages containing ε . See [UT Austin notes: language hierarchy and automata](#).

A Compiler Uses Several Kinds of Reasoning

Compilation translates source into a target representation while preserving the required behavior. The target may be assembly, machine code, bytecode, or another source language.



The front end analyzes source; later stages transform and emit code. Passes may interleave, with optimization at several representations. Execution is separate, though JIT compilation occurs during execution.

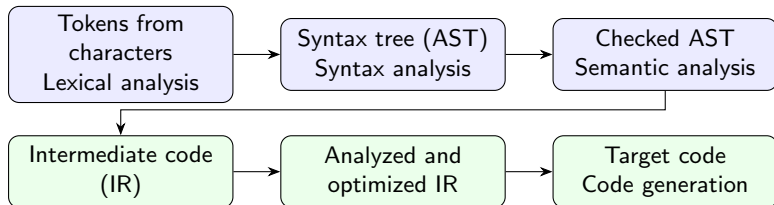
Not one Chomsky type per compiler phase

Regular token rules and CFG syntax are useful design choices. Semantic checking and optimization are algorithms over program data, not automatically Type 1 and Type 0 language recognizers.

Source: [Cornell CS 4120: anatomy of a compiler](#).

A Compiler Uses Several Kinds of Reasoning

Compilation translates source into a target representation while preserving the required behavior. The target may be assembly, machine code, bytecode, or another source language.



The front end analyzes source; later stages transform and emit code. Passes may interleave, with optimization at several representations. Execution is separate, though JIT compilation occurs during execution.

Not one Chomsky type per compiler phase

Regular token rules and CFG syntax are useful design choices. Semantic checking and optimization are algorithms over program data, not automatically Type 1 and Type 0 language recognizers.

Source: [Cornell CS 4120: anatomy of a compiler](#).

Lexing: Finite Memory, Not No Memory

In a small C-like language, the character sequence

`int count = 123;` $\mapsto$ INT, ID(count), ASSIGN, NUM(123), SEMI.

For example, digit strings use DD^* , where $D = \{0, \dots, 9\}$; identifiers may use $A(A \cup D)^*$, where A is a chosen letter set. A finite automaton remembers *which pattern prefix has been seen*.

Token boundaries and priorities matter

Under longest-match rules, `integer` is one identifier, not keyword `int` followed by `eger`. A keyword rule can win an equal-length tie for `int`. Lexeme storage and source positions are implementation bookkeeping beyond finite-state recognition.

Beyond compilers: fixed-format IDs, literal/regular-expression searches, and fixed byte signatures in network traffic are useful regular pattern tasks. A format match alone does not establish that an email is deliverable, a URL is reachable, or a packet is safe.

Tool connection: Lex/Flex-style scanners turn token patterns into scanning code. Not all language-specific lexing features are purely regular.

Source: Flex manual: longest match and rule priority.

Lexing: Finite Memory, Not No Memory

In a small C-like language, the character sequence

`int count = 123;` $\mapsto$ INT, ID(count), ASSIGN, NUM(123), SEMI.

For example, digit strings use DD^* , where $D = \{0, \dots, 9\}$; identifiers may use $A(A \cup D)^*$, where A is a chosen letter set. A finite automaton remembers *which pattern prefix has been seen*.

Token boundaries and priorities matter

Under longest-match rules, `integer` is one identifier, not keyword `int` followed by `eger`. A keyword rule can win an equal-length tie for `int`. Lexeme storage and source positions are implementation bookkeeping beyond finite-state recognition.

Beyond compilers: fixed-format IDs, literal/regular-expression searches, and fixed byte signatures in network traffic are useful regular pattern tasks. A format match alone does not establish that an email is deliverable, a URL is reachable, or a packet is safe.

Tool connection: Lex/Flex-style scanners turn token patterns into scanning code. Not all language-specific lexing features are purely regular.

Source: Flex manual: longest match and rule priority.

Lexing: Finite Memory, Not No Memory

In a small C-like language, the character sequence

`int count = 123;` $\mapsto$ INT, ID(count), ASSIGN, NUM(123), SEMI.

For example, digit strings use DD^* , where $D = \{0, \dots, 9\}$; identifiers may use $A(A \cup D)^*$, where A is a chosen letter set. A finite automaton remembers *which pattern prefix has been seen*.

Token boundaries and priorities matter

Under longest-match rules, `integer` is one identifier, not keyword `int` followed by `eger`. A keyword rule can win an equal-length tie for `int`. Lexeme storage and source positions are implementation bookkeeping beyond finite-state recognition.

Beyond compilers: fixed-format IDs, literal/regular-expression searches, and fixed byte signatures in network traffic are useful regular pattern tasks. A format match alone does not establish that an email is deliverable, a URL is reachable, or a packet is safe.

Tool connection: Lex/Flex-style scanners turn token patterns into scanning code. Not all language-specific lexing features are purely regular.

Source: [Flex manual: longest match and rule priority](#).

Parsing: A Stack for Unbounded Nesting

Token recognition cannot by itself check arbitrary nesting. A stack can remember pending delimiters and require the right kind of closer:

`[[{}]]` is properly nested, `[[{}]]` is not.

A CFG describes structure; the resulting parse tree or AST determines grouping, for example before arithmetic evaluation.

Applications and their boundaries

- **JSON:** objects and arrays nest recursively; grammar-based parsing checks syntax, not an application's schema or business rules.
- **XML:** nesting is only part of well-formedness; matching arbitrary start/end tag names is an additional explicit constraint.
- **HTML:** browser parsing includes specified error recovery and tree-building rules, not just checking balanced tags.
- **Natural language:** CFG phrase structure is a useful model of clauses, not a complete theory of meaning or translation.

Tools: Yacc/Bison generate parsers; grammar conflicts still matter.

Sources: [Bison](#); [JSON \(RFC 8259\)](#); [XML](#); [HTML parsing](#).

Parsing: A Stack for Unbounded Nesting

Token recognition cannot by itself check arbitrary nesting. A stack can remember pending delimiters and require the right kind of closer:

`[{}]` is properly nested, `[{}]` is not.

A CFG describes structure; the resulting parse tree or AST determines grouping, for example before arithmetic evaluation.

Applications and their boundaries

- **JSON:** objects and arrays nest recursively; grammar-based parsing checks syntax, not an application's schema or business rules.
- **XML:** nesting is only part of well-formedness; matching arbitrary start/end tag names is an additional explicit constraint.
- **HTML:** browser parsing includes specified error recovery and tree-building rules, not just checking balanced tags.
- **Natural language:** CFG phrase structure is a useful model of clauses, not a complete theory of meaning or translation.

Tools: Yacc/Bison generate parsers; grammar conflicts still matter.

Sources: [Bison](#); [JSON \(RFC 8259\)](#); [XML](#); [HTML parsing](#).

Parsing: A Stack for Unbounded Nesting

Token recognition cannot by itself check arbitrary nesting. A stack can remember pending delimiters and require the right kind of closer:

`[[{}]]` is properly nested, `[[{}]]` is not.

A CFG describes structure; the resulting parse tree or AST determines grouping, for example before arithmetic evaluation.

Applications and their boundaries

- **JSON:** objects and arrays nest recursively; grammar-based parsing checks syntax, not an application's schema or business rules.
- **XML:** nesting is only part of well-formedness; matching arbitrary start/end tag names is an additional explicit constraint.
- **HTML:** browser parsing includes specified error recovery and tree-building rules, not just checking balanced tags.
- **Natural language:** CFG phrase structure is a useful model of clauses, not a complete theory of meaning or translation.

Tools: Yacc/Bison generate parsers; grammar conflicts still matter.

Sources: [Bison](#); [JSON \(RFC 8259\)](#); [XML](#); [HTML parsing](#).

Semantic Checks: Structure Is Not Enough

Suppose a toy language uses the declaration rule $\text{Decl} \rightarrow \text{Type ID} = \text{Expr} ;$.

It allows integer and string literals as expressions but forbids assigning a string to an integer variable. Then `int x = "hello";` fits the syntax while violating its type rules.

What the semantic pass adds

Symbol tables record declarations and scopes; typing rules check expressions against expected types. Checks reject undeclared names, incompatible assignments, or forbidden duplicate declarations. Shadowing and overloading policies vary by language.

Two meanings of “context-sensitive”

Dependence on declarations is not a proof of Type 1 membership: that needs an encoding and a linear-space recognizer. Compilers use semantic rules and data structures, not normally a context-sensitive grammar parser.

Static checks do not guarantee termination or intended behavior. A game’s “key present and switch on” test is likewise a state predicate, not automatically a Type 1 language.

Reading: [Cornell CS 4120: semantic analysis and typing environments](#).

Semantic Checks: Structure Is Not Enough

Suppose a toy language uses the declaration rule $\text{Decl} \rightarrow \text{Type ID} = \text{Expr} ;$.

It allows integer and string literals as expressions but forbids assigning a string to an integer variable. Then `int x = "hello";` fits the syntax while violating its type rules.

What the semantic pass adds

Symbol tables record declarations and scopes; typing rules check expressions against expected types. Checks reject undeclared names, incompatible assignments, or forbidden duplicate declarations. Shadowing and overloading policies vary by language.

Two meanings of “context-sensitive”

Dependence on declarations is not a proof of Type 1 membership: that needs an encoding and a linear-space recognizer. Compilers use semantic rules and data structures, not normally a context-sensitive grammar parser.

Static checks do not guarantee termination or intended behavior. A game’s “key present and switch on” test is likewise a state predicate, not automatically a Type 1 language.

Reading: [Cornell CS 4120: semantic analysis and typing environments](#).

Semantic Checks: Structure Is Not Enough

Suppose a toy language uses the declaration rule $\text{Decl} \rightarrow \text{Type ID} = \text{Expr} ;$.

It allows integer and string literals as expressions but forbids assigning a string to an integer variable. Then `int x = "hello";` fits the syntax while violating its type rules.

What the semantic pass adds

Symbol tables record declarations and scopes; typing rules check expressions against expected types. Checks reject undeclared names, incompatible assignments, or forbidden duplicate declarations. Shadowing and overloading policies vary by language.

Two meanings of “context-sensitive”

Dependence on declarations is not a proof of Type 1 membership: that needs an encoding and a linear-space recognizer. Compilers use semantic rules and data structures, not normally a context-sensitive grammar parser.

Static checks do not guarantee termination or intended behavior. A game’s “key present and switch on” test is likewise a state predicate, not automatically a Type 1 language.

Reading: [Cornell CS 4120: semantic analysis and typing environments](#).

Beyond a Stack: Reusing Input-Sized Workspace

A concrete Type 1 example is

$$L = \{a^k b^k c^k : k \geq 1\}.$$

Our pumping argument shows L is not context-free. Yet a machine can decide it using only the input cells plus finite control.

A linear-bounded marking algorithm

- 1 Check that the input has the form $a^+ b^+ c^+$.
- 2 Mark the leftmost unmarked a , then one unmarked b , then one unmarked c . Reject if either partner is missing.
- 3 Return to the left and repeat. When no unmarked a remains, accept iff no unmarked b or c remains.

Use marked versions A, B, C of the input symbols; no extra tape region is needed. Each successful round removes one obligation from each block.

A stack accesses only its top; an LBA can revisit and mark earlier input. “Bounded” means linear in input length, not a fixed amount for all inputs. With finitely many configurations per input, configuration-graph reachability decides LBA acceptance. This example illustrates workspace, not a required compiler phase. Reading: [1, 3].

Beyond a Stack: Reusing Input-Sized Workspace

A concrete Type 1 example is

$$L = \{a^k b^k c^k : k \geq 1\}.$$

Our pumping argument shows L is not context-free. Yet a machine can decide it using only the input cells plus finite control.

A linear-bounded marking algorithm

- 1 Check that the input has the form $a^+ b^+ c^+$.
- 2 Mark the leftmost unmarked a , then one unmarked b , then one unmarked c . Reject if either partner is missing.
- 3 Return to the left and repeat. When no unmarked a remains, accept iff no unmarked b or c remains.

Use marked versions A, B, C of the input symbols; no extra tape region is needed. Each successful round removes one obligation from each block.

A stack accesses only its top; an LBA can revisit and mark earlier input. “Bounded” means linear in input length, not a fixed amount for all inputs. With finitely many configurations per input, configuration-graph reachability decides LBA acceptance. This example illustrates workspace, not a required compiler phase. Reading: [1, 3].

Beyond a Stack: Reusing Input-Sized Workspace

A concrete Type 1 example is

$$L = \{a^k b^k c^k : k \geq 1\}.$$

Our pumping argument shows L is not context-free. Yet a machine can decide it using only the input cells plus finite control.

A linear-bounded marking algorithm

- 1 Check that the input has the form $a^+ b^+ c^+$.
- 2 Mark the leftmost unmarked a , then one unmarked b , then one unmarked c . Reject if either partner is missing.
- 3 Return to the left and repeat. When no unmarked a remains, accept iff no unmarked b or c remains.

Use marked versions A, B, C of the input symbols; no extra tape region is needed. Each successful round removes one obligation from each block.

A stack accesses only its top; an LBA can revisit and mark earlier input. “Bounded” means linear in input length, not a fixed amount for all inputs. With finitely many configurations per input, configuration-graph reachability decides LBA acceptance. This example illustrates workspace, not a required compiler phase. Reading: [1, 3].

Beyond a Stack: Reusing Input-Sized Workspace

A concrete Type 1 example is

$$L = \{a^k b^k c^k : k \geq 1\}.$$

Our pumping argument shows L is not context-free. Yet a machine can decide it using only the input cells plus finite control.

A linear-bounded marking algorithm

- 1 Check that the input has the form $a^+ b^+ c^+$.
- 2 Mark the leftmost unmarked a , then one unmarked b , then one unmarked c . Reject if either partner is missing.
- 3 Return to the left and repeat. When no unmarked a remains, accept iff no unmarked b or c remains.

Use marked versions A, B, C of the input symbols; no extra tape region is needed. Each successful round removes one obligation from each block.

A stack accesses only its top; an LBA can revisit and mark earlier input. “Bounded” means linear in input length, not a fixed amount for all inputs. With finitely many configurations per input, configuration-graph reachability decides LBA acceptance.

This example illustrates workspace, not a required compiler phase. Reading: [1, 3].

General Computation: Power Comes with Limits

Type 0 grammars generate exactly the Turing-recognizable languages. A recognizer accepts members, but may run forever on nonmembers. Consider the language of encoded program–input pairs

$$H = \{\langle P, x \rangle : P \text{ eventually halts on } x\}.$$

Simulating $P(x)$ recognizes H : accept if the simulation halts. No algorithm decides H correctly for every pair and always terminates.

Why perfect reachability analysis would be impossible

Given P, x , construct a wrapper that first simulates $P(x)$, then executes a distinguished statement `emit(1)`. The statement is reachable exactly when $P(x)$ halts. An always-correct, always-terminating reachability checker for arbitrary programs would therefore decide the halting problem.

This is a limit on *all* algorithms, not merely on CFG parsers. Restricted cases can still be decided, and useful analyses can return conservative answers when they cannot establish a fact.

General-purpose does not mean omnipotent. AI and cryptography are application areas, not extra hierarchy levels; computational difficulty is not the same as undecidability.

Source: Cornell: [computability and limits of program analysis](#).

General Computation: Power Comes with Limits

Type 0 grammars generate exactly the Turing-recognizable languages. A recognizer accepts members, but may run forever on nonmembers. Consider the language of encoded program–input pairs

$$H = \{\langle P, x \rangle : P \text{ eventually halts on } x\}.$$

Simulating $P(x)$ recognizes H : accept if the simulation halts. No algorithm decides H correctly for every pair and always terminates.

Why perfect reachability analysis would be impossible

Given P, x , construct a wrapper that first simulates $P(x)$, then executes a distinguished statement `emit(1)`. The statement is reachable exactly when $P(x)$ halts. An always-correct, always-terminating reachability checker for arbitrary programs would therefore decide the halting problem.

This is a limit on *all* algorithms, not merely on CFG parsers. Restricted cases can still be decided, and useful analyses can return conservative answers when they cannot establish a fact.

General-purpose does not mean omnipotent. AI and cryptography are application areas, not extra hierarchy levels; computational difficulty is not the same as undecidability.

Source: Cornell: [computability and limits of program analysis](#).

General Computation: Power Comes with Limits

Type 0 grammars generate exactly the Turing-recognizable languages. A recognizer accepts members, but may run forever on nonmembers. Consider the language of encoded program–input pairs

$$H = \{\langle P, x \rangle : P \text{ eventually halts on } x\}.$$

Simulating $P(x)$ recognizes H : accept if the simulation halts. No algorithm decides H correctly for every pair and always terminates.

Why perfect reachability analysis would be impossible

Given P, x , construct a wrapper that first simulates $P(x)$, then executes a distinguished statement `emit(1)`. The statement is reachable exactly when $P(x)$ halts. An always-correct, always-terminating reachability checker for arbitrary programs would therefore decide the halting problem.

This is a limit on *all* algorithms, not merely on CFG parsers. Restricted cases can still be decided, and useful analyses can return conservative answers when they cannot establish a fact.

General-purpose does not mean omnipotent. AI and cryptography are application areas, not extra hierarchy levels; computational difficulty is not the same as undecidability.

Source: [Cornell: computability and limits of program analysis](#).

Optimization: Safe Rewrites, Not Omniscience

An optimizer seeks better time, space, or energy use while preserving required observable behavior. Useful rewrites need not solve arbitrary logic.

Constant folding: under exact integer arithmetic, replace $60 * 60 * 24$ with 86400. Machine types, overflow, and floating-point rules must be respected.

Unreachable code: remove the body of a literal `if (false)` branch. Removing an unused computation also requires checking side effects and traps.

Loop unrolling: a simple loop calling `work(i)` for $i = 0, 1, 2$ can become `work(0); work(1); work(2);` when the loop variable is local and the body cannot alter control flow. This may reduce loop overhead but increases code size; speedup is not guaranteed.

Separate legality from profitability

Sound analyses justify *whether* a rewrite preserves behavior; cost models and heuristics choose *whether it is worthwhile*. Uncertainty is not permission to change the program's meaning.

Course lesson: choose models to fit the constraints; recognize limits on universal analysis. Optimization is not a Type 0 classification.

Examples and implementation details: [LLVM transformation passes](#) and [MLIR canonicalization](#).

Optimization: Safe Rewrites, Not Omniscience

An optimizer seeks better time, space, or energy use while preserving required observable behavior. Useful rewrites need not solve arbitrary logic.

Constant folding: under exact integer arithmetic, replace `60 * 60 * 24` with `86400`. Machine types, overflow, and floating-point rules must be respected.

Unreachable code: remove the body of a literal `if (false)` branch. Removing an unused computation also requires checking side effects and traps.

Loop unrolling: a simple loop calling `work(i)` for $i = 0, 1, 2$ can become `work(0); work(1); work(2);` when the loop variable is local and the body cannot alter control flow. This may reduce loop overhead but increases code size; speedup is not guaranteed.

Separate legality from profitability

Sound analyses justify *whether* a rewrite preserves behavior; cost models and heuristics choose *whether it is worthwhile*. Uncertainty is not permission to change the program's meaning.

Course lesson: choose models to fit the constraints; recognize limits on universal analysis. Optimization is not a Type 0 classification.

Examples and implementation details: [LLVM transformation passes](#) and [MLIR canonicalization](#).

Optimization: Safe Rewrites, Not Omniscience

An optimizer seeks better time, space, or energy use while preserving required observable behavior. Useful rewrites need not solve arbitrary logic.

Constant folding: under exact integer arithmetic, replace `60 * 60 * 24` with `86400`. Machine types, overflow, and floating-point rules must be respected.

Unreachable code: remove the body of a literal `if (false)` branch. Removing an unused computation also requires checking side effects and traps.

Loop unrolling: a simple loop calling `work(i)` for $i = 0, 1, 2$ can become `work(0); work(1); work(2);` when the loop variable is local and the body cannot alter control flow. This may reduce loop overhead but increases code size; speedup is not guaranteed.

Separate legality from profitability

Sound analyses justify *whether* a rewrite preserves behavior; cost models and heuristics choose *whether it is worthwhile*. Uncertainty is not permission to change the program's meaning.

Course lesson: choose models to fit the constraints; recognize limits on universal analysis. Optimization is not a Type 0 classification.

Examples and implementation details: [LLVM transformation passes](#) and [MLIR canonicalization](#).

Optimization: Safe Rewrites, Not Omniscience

An optimizer seeks better time, space, or energy use while preserving required observable behavior. Useful rewrites need not solve arbitrary logic.

Constant folding: under exact integer arithmetic, replace `60 * 60 * 24` with `86400`. Machine types, overflow, and floating-point rules must be respected.

Unreachable code: remove the body of a literal `if (false)` branch. Removing an unused computation also requires checking side effects and traps.

Loop unrolling: a simple loop calling `work(i)` for $i = 0, 1, 2$ can become `work(0); work(1); work(2);` when the loop variable is local and the body cannot alter control flow. This may reduce loop overhead but increases code size; speedup is not guaranteed.

Separate legality from profitability

Sound analyses justify *whether* a rewrite preserves behavior; cost models and heuristics choose *whether it is worthwhile*. Uncertainty is not permission to change the program's meaning.

Course lesson: choose models to fit the constraints; recognize limits on universal analysis. Optimization is not a Type 0 classification.

Examples and implementation details: [LLVM transformation passes](#) and [MLIR canonicalization](#).

- ① $S \Rightarrow 0S1 \Rightarrow 00S11 \Rightarrow 000S111 \Rightarrow 000111$.
- ② The first expansion $S \Rightarrow 0S0$ gives the requested partial tree. Complete it with the inner production $S \rightarrow 1S1$, then $S \rightarrow \varepsilon$.
- ③ $S \rightarrow AB, A \rightarrow 0A \mid \varepsilon, B \rightarrow 1B \mid \varepsilon$.
- ④ Two leftmost derivations are $S \Rightarrow SS \Rightarrow SSS \Rightarrow aSS \Rightarrow aaS \Rightarrow aaa$ and $S \Rightarrow SS \Rightarrow aS \Rightarrow aSS \Rightarrow aaS \Rightarrow aaa$.
- ⑤ Keep $T \rightarrow T \times F \mid F$ and use $F \rightarrow A \uparrow F \mid A, A \rightarrow (E) \mid \text{id}$. The right recursion makes exponentiation right-associative.
- ⑥ Use the invariant $a^k S b^k$ for soundness and apply the recursive rule exactly n times for completeness.

Hints: Simplifying CFGs

- 1 A and B are nullable; C and S are not. Replace $S \rightarrow ABC$ by

$$S \rightarrow ABC \mid BC \mid AC \mid C.$$

- 2 The unit closures allow S to inherit the non-unit rules of both A and B , and A to inherit those of B :

$$S \rightarrow d \mid a \mid bB \mid c, \quad A \rightarrow a \mid bB \mid c, \quad B \rightarrow bB \mid c.$$

- 3 D is non-generating. After it is removed, C is generating but unreachable from S . Thus both C and D are useless.
- 4 Removing non-generating symbols first can remove productions and expose additional reachability facts. The second pass then explores only productions capable of participating in terminal derivations.

Hints: CNF, GNF, and s-Grammars

- ① Only $A \rightarrow BC$ and $A \rightarrow a$ have CNF form.
- ② Add $S_0 \rightarrow S$, then eliminate the nullable rule to obtain $S_0 \rightarrow S \mid \varepsilon$ and $S \rightarrow aSb \mid ab$. Introduce $A \rightarrow a$, $B \rightarrow b$, and $X \rightarrow SB$. After eliminating $S_0 \rightarrow S$, use

$$S_0 \rightarrow AX \mid AB \mid \varepsilon, \quad S \rightarrow AX \mid AB, \quad X \rightarrow SB, \quad A \rightarrow a, \quad B \rightarrow b.$$

- ③ Substitute $A \rightarrow aA \mid a$ into $S \rightarrow AB$ to obtain

$$S \rightarrow aAB \mid aB \mid b, \quad A \rightarrow aA \mid a, \quad B \rightarrow bB \mid b.$$

Every rule now begins with a terminal and is followed only by variables.

- ④ The two S -productions share the same leading terminal a ; the pair (S, a) therefore does not select a unique production.
- ⑤ Each GNF production introduces exactly one terminal at the left edge of the remaining sentential form. Starting with zero terminals and ending with n therefore requires exactly n steps.
- ⑥ The length-one cells are $\{A\}, \{B\}, \{A\}, \{B\}$. The two occurrences of ab produce $\{S\}$, but neither length-three cell nor the length-four top cell gains a variable. Therefore CYK rejects $abab$.

Hints: CFL Properties

- ① $L \cup R$, $L \cap R$, and $L \setminus R$ are guaranteed CFLs; $R \setminus L$ is not guaranteed context-free.
- ② Use fresh S with $S \rightarrow S_1 \mid S_2$, $S \rightarrow S_1 S_2$, or $S \rightarrow S_1 S \mid \varepsilon$, respectively.
- ③ Take the union of a grammar for $a^n b^n c^*$ and one for $a^* b^n c^n$.
- ④ If complement closure held, then $L_1 \cap L_2 = \overline{\overline{L_1} \cup \overline{L_2}}$ would be a CFL.
- ⑤ Membership, emptiness, and finiteness are decidable; universality, equivalence, and ambiguity are undecidable.
- ⑥ Nonclosure means that at least one counterexample pair exists, not that every pair fails.

Hints: CFL Pumping Exercises

- 1 Choose $s = 0^p 1^p 0^p$. The bounded window cannot touch both outer 0-blocks. Pumping down alters at most two adjacent blocks, so the three required block lengths cannot remain equal.
- 2 Choose $s = a^{p^3}$. If $d = |v| + |y|$, then $1 \leq d \leq p$, while $(p+1)^3 - p^3 = 3p^2 + 3p + 1 > p$; pumping up lands between consecutive cubes.
- 3 The opponent, not the student, chooses v and y . Every legal decomposition must be handled.
- 4 Either v or y may be empty, but $|vy| > 0$ forbids both from being empty.
- 5 The lemma promises the existence of some pumpable decomposition for every sufficiently long string; one successful split does not establish that universal condition.
- 6 Touching an a and a c would also include all p intervening b 's, giving length at least $p + 2$.

Deterministic CFLs Behave Differently

A **deterministic context-free language** (DCFL) is accepted by a deterministic pushdown automaton. Determinism changes several familiar properties.

Property	CFL	DCFL
complement	not closed	closed
union and intersection	union only	not closed
intersection with regular languages	closed	closed
has an unambiguous grammar	not always	always

Thus

$$\text{DCFL} \subsetneq \text{unambiguous CFL} \subsetneq \text{CFL}.$$

An inherently ambiguous CFL therefore cannot be deterministic.

Technical caution

Complementing a DPDA requires a normalized machine that consumes the entire input and does not diverge forever on ϵ -moves; merely swapping final states is not sufficient for an arbitrary presentation.

Deterministic CFLs Behave Differently

A **deterministic context-free language** (DCFL) is accepted by a deterministic pushdown automaton. Determinism changes several familiar properties.

Property	CFL	DCFL
complement	not closed	closed
union and intersection	union only	not closed
intersection with regular languages	closed	closed
has an unambiguous grammar	not always	always

Thus

$$\text{DCFL} \subsetneq \text{unambiguous CFL} \subsetneq \text{CFL}.$$

An inherently ambiguous CFL therefore cannot be deterministic.

Technical caution

Complementing a DPDA requires a normalized machine that consumes the entire input and does not diverge forever on ϵ -moves; merely swapping final states is not sufficient for an arbitrary presentation.

Deterministic CFLs Behave Differently

A **deterministic context-free language** (DCFL) is accepted by a deterministic pushdown automaton. Determinism changes several familiar properties.

Property	CFL	DCFL
complement	not closed	closed
union and intersection	union only	not closed
intersection with regular languages	closed	closed
has an unambiguous grammar	not always	always

Thus

$$\text{DCFL} \subsetneq \text{unambiguous CFL} \subsetneq \text{CFL}.$$

An inherently ambiguous CFL therefore cannot be deterministic.

Technical caution

Complementing a DPDA requires a normalized machine that consumes the entire input and does not diverge forever on ϵ -moves; merely swapping final states is not sufficient for an arbitrary presentation.

Ogden's Lemma: Mark the Important Positions

The ordinary CFL pumping lemma lets the opponent hide the pumped pieces in an unhelpful region. Ogden's lemma strengthens it by letting us mark the positions that matter.

Ogden's Lemma

For every CFL L , some $p \geq 1$ has the following property. If $z \in L$ has at least p marked positions, then $z = uvwxy$ so that

- 1 v and x together contain at least one marked position;
- 2 vwx contains at most p marked positions; and
- 3 $uv^iwx^iy \in L$ for every $i \geq 0$.

Marking every position recovers the usual pumping lemma. Strategic marking can force at least one pumped part into a selected block, making some non-CFL and inherent-ambiguity proofs possible when ordinary pumping is too weak.

Original paper: W. Ogden, "A Helpful Result for Proving Inherent Ambiguity" (1968).

Ogden's Lemma: Mark the Important Positions

The ordinary CFL pumping lemma lets the opponent hide the pumped pieces in an unhelpful region. Ogden's lemma strengthens it by letting us mark the positions that matter.

Ogden's Lemma

For every CFL L , some $p \geq 1$ has the following property. If $z \in L$ has at least p marked positions, then $z = uvwxy$ so that

- 1 v and x together contain at least one marked position;
- 2 vwx contains at most p marked positions; and
- 3 $uv^iwx^iy \in L$ for every $i \geq 0$.

Marking every position recovers the usual pumping lemma. Strategic marking can force at least one pumped part into a selected block, making some non-CFL and inherent-ambiguity proofs possible when ordinary pumping is too weak.

Original paper: W. Ogden, "A Helpful Result for Proving Inherent Ambiguity" (1968).

Ogden's Lemma: Mark the Important Positions

The ordinary CFL pumping lemma lets the opponent hide the pumped pieces in an unhelpful region. Ogden's lemma strengthens it by letting us mark the positions that matter.

Ogden's Lemma

For every CFL L , some $p \geq 1$ has the following property. If $z \in L$ has at least p marked positions, then $z = uvwxy$ so that

- 1 v and x together contain at least one marked position;
- 2 vwx contains at most p marked positions; and
- 3 $uv^iwx^iy \in L$ for every $i \geq 0$.

Marking every position recovers the usual pumping lemma. Strategic marking can force at least one pumped part into a selected block, making some non-CFL and inherent-ambiguity proofs possible when ordinary pumping is too weak.

Original paper: W. Ogden, "A Helpful Result for Proving Inherent Ambiguity" (1968).

Parikh's Theorem: Counts Forget Order

For $\Sigma = \{a_1, \dots, a_k\}$, the **Parikh image** of a word is

$$\Psi(w) = (|w|_{a_1}, \dots, |w|_{a_k}).$$

It records symbol counts but discards their order.

Parikh's Theorem

The Parikh image of every context-free language is a semilinear set: a finite union of sets of the form

$$\mathbf{b} + n_1 \mathbf{p}_1 + \dots + n_r \mathbf{p}_r, \text{ where } n_i \geq 0.$$

Equivalently, every CFL has the same Parikh image as some regular language.

Therefore a language whose count vectors are not semilinear cannot be context-free.

The converse is false: semilinear counts do not capture word order.

Original paper: R. Parikh, "On Context-Free Languages" (1966).

Parikh's Theorem: Counts Forget Order

For $\Sigma = \{a_1, \dots, a_k\}$, the **Parikh image** of a word is

$$\Psi(w) = (|w|_{a_1}, \dots, |w|_{a_k}).$$

It records symbol counts but discards their order.

Parikh's Theorem

The Parikh image of every context-free language is a semilinear set: a finite union of sets of the form

$$\mathbf{b} + n_1 \mathbf{p}_1 + \dots + n_r \mathbf{p}_r, n_i \geq 0.$$

Equivalently, every CFL has the same Parikh image as some regular language.

Therefore a language whose count vectors are not semilinear cannot be context-free.

The converse is false: semilinear counts do not capture word order.

Original paper: R. Parikh, "On Context-Free Languages" (1966).

Parikh's Theorem: Counts Forget Order

For $\Sigma = \{a_1, \dots, a_k\}$, the **Parikh image** of a word is

$$\Psi(w) = (|w|_{a_1}, \dots, |w|_{a_k}).$$

It records symbol counts but discards their order.

Parikh's Theorem

The Parikh image of every context-free language is a semilinear set: a finite union of sets of the form

$$\mathbf{b} + n_1 \mathbf{p}_1 + \dots + n_r \mathbf{p}_r, n_i \geq 0.$$

Equivalently, every CFL has the same Parikh image as some regular language.

Therefore a language whose count vectors are not semilinear cannot be context-free. The converse is false: semilinear counts do not capture word order.

Original paper: R. Parikh, "On Context-Free Languages" (1966).

Chomsky–Schützenberger Representation

Let D_k be the Dyck language of correctly nested strings using k types of parentheses.

Representation Theorem

A language L is context-free if and only if there exist some k , a regular language R , and a homomorphism h such that

$$L = h(D_k \cap R).$$

This decomposes every CFL into three ingredients:

- 1 a Dyck language supplies unbounded nested memory;
- 2 a regular filter R restricts the permitted structural patterns;
- 3 a homomorphism translates the surviving brackets into terminals.

Conceptual significance

Balanced nesting is the universal context-free mechanism; finite-state control and symbol translation customize it into a particular CFL.

Chomsky and Schützenberger's foundational treatment.

Chomsky–Schützenberger Representation

Let D_k be the Dyck language of correctly nested strings using k types of parentheses.

Representation Theorem

A language L is context-free if and only if there exist some k , a regular language R , and a homomorphism h such that

$$L = h(D_k \cap R).$$

This decomposes every CFL into three ingredients:

- 1 a Dyck language supplies unbounded nested memory;
- 2 a regular filter R restricts the permitted structural patterns;
- 3 a homomorphism translates the surviving brackets into terminals.

Conceptual significance

Balanced nesting is the universal context-free mechanism; finite-state control and symbol translation customize it into a particular CFL.

Chomsky and Schützenberger's foundational treatment.

Chomsky–Schützenberger Representation

Let D_k be the Dyck language of correctly nested strings using k types of parentheses.

Representation Theorem

A language L is context-free if and only if there exist some k , a regular language R , and a homomorphism h such that

$$L = h(D_k \cap R).$$

This decomposes every CFL into three ingredients:

- 1 a Dyck language supplies unbounded nested memory;
- 2 a regular filter R restricts the permitted structural patterns;
- 3 a homomorphism translates the surviving brackets into terminals.

Conceptual significance

Balanced nesting is the universal context-free mechanism; finite-state control and symbol translation customize it into a particular CFL.

Chomsky and Schützenberger's foundational treatment.

Substitution: A Powerful Closure Principle

A **substitution** assigns each terminal $a \in \Sigma$ a language $\sigma(a)$ and extends to words by concatenation and to languages by union.

Closure under context-free substitution

If L is context-free and every $\sigma(a)$ is context-free, then $\sigma(L)$ is context-free.

Grammar construction. Rename all variables to be disjoint. In a grammar for L , replace each terminal a by the start variable S_a of a grammar for $\sigma(a)$, and include all those productions.

Example

Let $L = \{0^n 1^n : n \geq 0\}$, $\sigma(0) = \{a^i b^i : i \geq 1\}$, and $\sigma(1) = \{c, d\}$. Then $\sigma(L)$ consists of n matched $a^i b^i$ blocks followed by exactly n symbols chosen from $\{c, d\}$, and is context-free by the theorem.

Homomorphism closure is the special case in which every $\sigma(a)$ is a singleton language.

Substitution: A Powerful Closure Principle

A **substitution** assigns each terminal $a \in \Sigma$ a language $\sigma(a)$ and extends to words by concatenation and to languages by union.

Closure under context-free substitution

If L is context-free and every $\sigma(a)$ is context-free, then $\sigma(L)$ is context-free.

Grammar construction. Rename all variables to be disjoint. In a grammar for L , replace each terminal a by the start variable S_a of a grammar for $\sigma(a)$, and include all those productions.

Example

Let $L = \{0^n 1^n : n \geq 0\}$, $\sigma(0) = \{a^i b^i : i \geq 1\}$, and $\sigma(1) = \{c, d\}$. Then $\sigma(L)$ consists of n matched $a^i b^i$ blocks followed by exactly n symbols chosen from $\{c, d\}$, and is context-free by the theorem.

Homomorphism closure is the special case in which every $\sigma(a)$ is a singleton language.

Substitution: A Powerful Closure Principle

A **substitution** assigns each terminal $a \in \Sigma$ a language $\sigma(a)$ and extends to words by concatenation and to languages by union.

Closure under context-free substitution

If L is context-free and every $\sigma(a)$ is context-free, then $\sigma(L)$ is context-free.

Grammar construction. Rename all variables to be disjoint. In a grammar for L , replace each terminal a by the start variable S_a of a grammar for $\sigma(a)$, and include all those productions.

Example

Let $L = \{0^n 1^n : n \geq 0\}$, $\sigma(0) = \{a^i b^i : i \geq 1\}$, and $\sigma(1) = \{c, d\}$. Then $\sigma(L)$ consists of n matched $a^i b^i$ blocks followed by exactly n symbols chosen from $\{c, d\}$, and is context-free by the theorem.

Homomorphism closure is the special case in which every $\sigma(a)$ is a singleton language.

A Surprising Decidability Boundary

For arbitrary CFGs, language equivalence is undecidable. Restricting the machines to deterministic pushdown automata changes the answer.

Sénizergues' Theorem

Equivalence of deterministic pushdown automata is decidable: there is an algorithm that determines whether two DPDAs accept the same language.

The result is much deeper than DFA equivalence. It does not imply a simple product-and-reachability test, and it does not extend to arbitrary nondeterministic PDAs.

Lesson

Two models can recognize closely related classes yet have dramatically different meta-problems. Determinism affects not only parsing speed but also what questions about machines are decidable.

G. Sénizergues, "The Equivalence Problem for Deterministic Pushdown Automata Is Decidable" (1997).

A Surprising Decidability Boundary

For arbitrary CFGs, language equivalence is undecidable. Restricting the machines to deterministic pushdown automata changes the answer.

Sénizergues' Theorem

Equivalence of deterministic pushdown automata is decidable: there is an algorithm that determines whether two DPDAs accept the same language.

The result is much deeper than DFA equivalence. It does not imply a simple product-and-reachability test, and it does not extend to arbitrary nondeterministic PDAs.

Lesson

Two models can recognize closely related classes yet have dramatically different meta-problems. Determinism affects not only parsing speed but also what questions about machines are decidable.

G. Sénizergues, "The Equivalence Problem for Deterministic Pushdown Automata Is Decidable" (1997).

A Surprising Decidability Boundary

For arbitrary CFGs, language equivalence is undecidable. Restricting the machines to deterministic pushdown automata changes the answer.

Sénizergues' Theorem

Equivalence of deterministic pushdown automata is decidable: there is an algorithm that determines whether two DPDAs accept the same language.

The result is much deeper than DFA equivalence. It does not imply a simple product-and-reachability test, and it does not extend to arbitrary nondeterministic PDAs.

Lesson

Two models can recognize closely related classes yet have dramatically different meta-problems. Determinism affects not only parsing speed but also what questions about machines are decidable.

G. Sénizergues, "The Equivalence Problem for Deterministic Pushdown Automata Is Decidable" (1997).

CNF Gives Quantitative Parse-Tree Bounds

Suppose an ε -free grammar is in CNF and a derivation produces a word of length $n \geq 1$.

Exact node and step counts

Its parse tree has n terminal leaves, $n - 1$ binary variable nodes, and therefore exactly

$$2n - 1$$

production applications: $n - 1$ rules of the form $A \rightarrow BC$ and n rules of the form $A \rightarrow a$.

If a CNF grammar with k variables generates any nonempty word, then it generates one of length at most 2^{k-1} . Choose a shortest word: no root-to-leaf path in its parse tree can repeat a variable, or the repeated portion could be removed to obtain a shorter word. Thus the variable height is at most k , and a binary tree of that height has at most 2^{k-1} leaves.

CNF Gives Quantitative Parse-Tree Bounds

Suppose an ε -free grammar is in CNF and a derivation produces a word of length $n \geq 1$.

Exact node and step counts

Its parse tree has n terminal leaves, $n - 1$ binary variable nodes, and therefore exactly

$$2n - 1$$

production applications: $n - 1$ rules of the form $A \rightarrow BC$ and n rules of the form $A \rightarrow a$.

If a CNF grammar with k variables generates any nonempty word, then it generates one of length at most 2^{k-1} . Choose a shortest word: no root-to-leaf path in its parse tree can repeat a variable, or the repeated portion could be removed to obtain a shorter word. Thus the variable height is at most k , and a binary tree of that height has at most 2^{k-1} leaves.

CNF Gives Quantitative Parse-Tree Bounds

Suppose an ε -free grammar is in CNF and a derivation produces a word of length $n \geq 1$.

Exact node and step counts

Its parse tree has n terminal leaves, $n - 1$ binary variable nodes, and therefore exactly

$$2n - 1$$

production applications: $n - 1$ rules of the form $A \rightarrow BC$ and n rules of the form $A \rightarrow a$.

If a CNF grammar with k variables generates any nonempty word, then it generates one of length at most 2^{k-1} . Choose a shortest word: no root-to-leaf path in its parse tree can repeat a variable, or the repeated portion could be removed to obtain a shorter word. Thus the variable height is at most k , and a binary tree of that height has at most 2^{k-1} leaves.

Beyond One Word: The Interchange Lemma

Pumping lemmas analyze repeatable structure inside one sufficiently long word. The **interchange lemma** instead considers a large collection of equal-length words in a CFL.

Informal form

Within a sufficiently large collection, many words can be decomposed with corresponding middle regions of the same length so that exchanging those regions between selected words still produces words in the language.

It is useful when every individual word appears pumpable but the language cannot support the many cross-combinations forced by context-free parse-tree structure. Like pumping and Ogden's lemma, it is a necessary condition, not a complete characterization of CFLs.

When to investigate it

The method is especially relevant for dense families involving copying, repetition, or many correlated choices where a single-word argument does not expose the contradiction.

Ogden, Ross, and Winklmann, "An Interchange Lemma for Context-Free Languages" (1985).

Beyond One Word: The Interchange Lemma

Pumping lemmas analyze repeatable structure inside one sufficiently long word. The **interchange lemma** instead considers a large collection of equal-length words in a CFL.

Informal form

Within a sufficiently large collection, many words can be decomposed with corresponding middle regions of the same length so that exchanging those regions between selected words still produces words in the language.

It is useful when every individual word appears pumpable but the language cannot support the many cross-combinations forced by context-free parse-tree structure. Like pumping and Ogden's lemma, it is a necessary condition, not a complete characterization of CFLs.

When to investigate it

The method is especially relevant for dense families involving copying, repetition, or many correlated choices where a single-word argument does not expose the contradiction.

Ogden, Ross, and Winklmann, "An Interchange Lemma for Context-Free Languages" (1985).

Beyond One Word: The Interchange Lemma

Pumping lemmas analyze repeatable structure inside one sufficiently long word. The **interchange lemma** instead considers a large collection of equal-length words in a CFL.

Informal form

Within a sufficiently large collection, many words can be decomposed with corresponding middle regions of the same length so that exchanging those regions between selected words still produces words in the language.

It is useful when every individual word appears pumpable but the language cannot support the many cross-combinations forced by context-free parse-tree structure. Like pumping and Ogden's lemma, it is a necessary condition, not a complete characterization of CFLs.

When to investigate it

The method is especially relevant for dense families involving copying, repetition, or many correlated choices where a single-word argument does not expose the contradiction.

Ogden, Ross, and Winklmann, "An Interchange Lemma for Context-Free Languages" (1985).

One Marker Can Change Deterministic Parsability

Compare the palindrome languages

$$P = \{ww^R : w \in \{0,1\}^*\}, \quad P_{\#} = \{w\#w^R : w \in \{0,1\}^*\}.$$

Both are context-free.

- $P_{\#}$ is deterministic context-free: push symbols before the marker, then pop and compare after the marker.
- P is not deterministic context-free: without a marker, a one-stack deterministic machine cannot know when to switch from pushing to matching.

The delimiter does not add unbounded information; it supplies one decisive structural event. This is why explicit separators and closing tokens often make programming-language syntax easier to parse deterministically.

One Marker Can Change Deterministic Parsability

Compare the palindrome languages

$$P = \{ww^R : w \in \{0,1\}^*\}, \quad P_{\#} = \{w\#w^R : w \in \{0,1\}^*\}.$$

Both are context-free.

- $P_{\#}$ is deterministic context-free: push symbols before the marker, then pop and compare after the marker.
- P is not deterministic context-free: without a marker, a one-stack deterministic machine cannot know when to switch from pushing to matching.

The delimiter does not add unbounded information; it supplies one decisive structural event. This is why explicit separators and closing tokens often make programming-language syntax easier to parse deterministically.

One Marker Can Change Deterministic Parsability

Compare the palindrome languages

$$P = \{ww^R : w \in \{0,1\}^*\}, \quad P_{\#} = \{w\#w^R : w \in \{0,1\}^*\}.$$

Both are context-free.

- $P_{\#}$ is deterministic context-free: push symbols before the marker, then pop and compare after the marker.
- P is not deterministic context-free: without a marker, a one-stack deterministic machine cannot know when to switch from pushing to matching.

The delimiter does not add unbounded information; it supplies one decisive structural event. This is why explicit separators and closing tokens often make programming-language syntax easier to parse deterministically.

The Same Counts Can Hide Different Complexity

Consider

$$L_1 = (abc)^*, \quad L_2 = \{a^n b^n c^n : n \geq 0\}.$$

The first language is regular; the second is not context-free. Nevertheless they have exactly the same Parikh image:

$$\Psi(L_1) = \Psi(L_2) = \{(n, n, n) : n \geq 0\}.$$

What Parikh's theorem cannot see

Semilinearity is a necessary condition for context-freeness, not a sufficient one. The obstruction in L_2 is the ordered three-way dependency, not the set of possible symbol counts.

This also explains why counting abstractions used in program analysis can prove some impossibility results but may lose essential sequencing information.

The Same Counts Can Hide Different Complexity

Consider

$$L_1 = (abc)^*, \quad L_2 = \{a^n b^n c^n : n \geq 0\}.$$

The first language is regular; the second is not context-free. Nevertheless they have exactly the same Parikh image:

$$\Psi(L_1) = \Psi(L_2) = \{(n, n, n) : n \geq 0\}.$$

What Parikh's theorem cannot see

Semilinearity is a necessary condition for context-freeness, not a sufficient one. The obstruction in L_2 is the ordered three-way dependency, not the set of possible symbol counts.

This also explains why counting abstractions used in program analysis can prove some impossibility results but may lose essential sequencing information.

The Same Counts Can Hide Different Complexity

Consider

$$L_1 = (abc)^*, \quad L_2 = \{a^n b^n c^n : n \geq 0\}.$$

The first language is regular; the second is not context-free. Nevertheless they have exactly the same Parikh image:

$$\Psi(L_1) = \Psi(L_2) = \{(n, n, n) : n \geq 0\}.$$

What Parikh's theorem cannot see

Semilinearity is a necessary condition for context-freeness, not a sufficient one. The obstruction in L_2 is the ordered three-way dependency, not the set of possible symbol counts.

This also explains why counting abstractions used in program analysis can prove some impossibility results but may lose essential sequencing information.

Nested Dependencies Work; Crossed Ones Fail

Compare

$$N = \{a^n b^m c^m d^n : n, m \geq 0\}$$

with

$$X = \{a^n b^m c^n d^m : n, m \geq 0\}.$$

N is context-free, generated by

$$S \rightarrow aSd \mid T, \quad T \rightarrow bTc \mid \varepsilon.$$

The a/d dependency surrounds the b/c dependency, matching stack order. The language X is not context-free: the required pairs cross. A stack that stores information for the a/c comparison blocks access to the information needed later for b/d in the wrong order.

Structural heuristic

A single stack naturally supports properly nested obligations. Independent crossed obligations are a warning sign that one stack is insufficient.

Nested Dependencies Work; Crossed Ones Fail

Compare

$$N = \{a^n b^m c^m d^n : n, m \geq 0\}$$

with

$$X = \{a^n b^m c^n d^m : n, m \geq 0\}.$$

N is context-free, generated by

$$S \rightarrow aSd \mid T, \quad T \rightarrow bTc \mid \varepsilon.$$

The a/d dependency surrounds the b/c dependency, matching stack order. The language X is not context-free: the required pairs cross. A stack that stores information for the a/c comparison blocks access to the information needed later for b/d in the wrong order.

Structural heuristic

A single stack naturally supports properly nested obligations. Independent crossed obligations are a warning sign that one stack is insufficient.

Nested Dependencies Work; Crossed Ones Fail

Compare

$$N = \{a^n b^m c^m d^n : n, m \geq 0\}$$

with

$$X = \{a^n b^m c^n d^m : n, m \geq 0\}.$$

N is context-free, generated by

$$S \rightarrow aSd \mid T, \quad T \rightarrow bTc \mid \varepsilon.$$

The a/d dependency surrounds the b/c dependency, matching stack order. The language X is not context-free: the required pairs cross. A stack that stores information for the a/c comparison blocks access to the information needed later for b/d in the wrong order.

Structural heuristic

A single stack naturally supports properly nested obligations. Independent crossed obligations are a warning sign that one stack is insufficient.

Nonclosure Does Not Mean “Never”

CFLs are not closed under complement, but particular CFLs can still have context-free complements. Let

$$L = \{a^n b^n : n \geq 0\} \subseteq \{a, b\}^*.$$

Then

$$\bar{L} = (\Sigma^* \setminus a^* b^*) \cup \{a^i b^j : 0 \leq i < j\} \cup \{a^i b^j : 0 \leq j < i\}.$$

The first component is regular, and the two unequal-count components are context-free; union closure makes $\bar{L}$ context-free.

Logical distinction

“The class is not closed under complement” means that at least one CFL has a non-CFL complement. It does not mean every complemented CFL leaves the class.

Nonclosure Does Not Mean “Never”

CFLs are not closed under complement, but particular CFLs can still have context-free complements. Let

$$L = \{a^n b^n : n \geq 0\} \subseteq \{a, b\}^*.$$

Then

$$\bar{L} = (\Sigma^* \setminus a^* b^*) \cup \{a^i b^j : 0 \leq i < j\} \cup \{a^i b^j : 0 \leq j < i\}.$$

The first component is regular, and the two unequal-count components are context-free; union closure makes $\bar{L}$ context-free.

Logical distinction

“The class is not closed under complement” means that at least one CFL has a non-CFL complement. It does not mean every complemented CFL leaves the class.

Nonclosure Does Not Mean “Never”

CFLs are not closed under complement, but particular CFLs can still have context-free complements. Let

$$L = \{a^n b^n : n \geq 0\} \subseteq \{a, b\}^*.$$

Then

$$\bar{L} = (\Sigma^* \setminus a^* b^*) \cup \{a^i b^j : 0 \leq i < j\} \cup \{a^i b^j : 0 \leq j < i\}.$$

The first component is regular, and the two unequal-count components are context-free; union closure makes $\bar{L}$ context-free.

Logical distinction

“The class is not closed under complement” means that at least one CFL has a non-CFL complement. It does not mean every complemented CFL leaves the class.

Another Classical Inherently Ambiguous CFL

Define

$$K_1 = \{a^n b^m c^m d^n : n, m \geq 1\},$$

$$K_2 = \{a^n b^n c^m d^m : n, m \geq 1\},$$

$$K = K_1 \cup K_2.$$

K_1 is context-free because its dependencies are nested; K_2 is the concatenation of two matched-block CFLs. Hence K is context-free. Their overlap is

$$K_1 \cap K_2 = \{a^n b^n c^n d^n : n \geq 1\}.$$

The language K is a standard second example of an inherently ambiguous CFL: every grammar for it must assign multiple parses to some words.

Do not use an invalid shortcut

Showing that two convenient component grammars overlap proves their union grammar ambiguous, but does not by itself prove the language inherently ambiguous. The unavoidable-ambiguity result requires a deeper theorem.

Another Classical Inherently Ambiguous CFL

Define

$$K_1 = \{a^n b^m c^m d^n : n, m \geq 1\},$$

$$K_2 = \{a^n b^n c^m d^m : n, m \geq 1\},$$

$$K = K_1 \cup K_2.$$

K_1 is context-free because its dependencies are nested; K_2 is the concatenation of two matched-block CFLs. Hence K is context-free. Their overlap is

$$K_1 \cap K_2 = \{a^n b^n c^n d^n : n \geq 1\}.$$

The language K is a standard second example of an inherently ambiguous CFL: every grammar for it must assign multiple parses to some words.

Do not use an invalid shortcut

Showing that two convenient component grammars overlap proves their union grammar ambiguous, but does not by itself prove the language inherently ambiguous. The unavoidable-ambiguity result requires a deeper theorem.

Another Classical Inherently Ambiguous CFL

Define

$$K_1 = \{a^n b^m c^m d^n : n, m \geq 1\},$$

$$K_2 = \{a^n b^n c^m d^m : n, m \geq 1\},$$

$$K = K_1 \cup K_2.$$

K_1 is context-free because its dependencies are nested; K_2 is the concatenation of two matched-block CFLs. Hence K is context-free. Their overlap is

$$K_1 \cap K_2 = \{a^n b^n c^n d^n : n \geq 1\}.$$

The language K is a standard second example of an inherently ambiguous CFL: every grammar for it must assign multiple parses to some words.

Do not use an invalid shortcut

Showing that two convenient component grammars overlap proves their union grammar ambiguous, but does not by itself prove the language inherently ambiguous. The unavoidable-ambiguity result requires a deeper theorem.

Maurer's Example and Unbounded Ambiguity

A structurally different inherently ambiguous language is

$$M = \{a^n b^m a^r b^m : n, m, r \geq 1\} \\ \cup \{a^n b^m a^n b^s : n, m, s \geq 1\}.$$

The first component matches the two b -blocks; the second matches the two a -blocks. Their shared words include $(a^n b^m)^2$. Maurer gave a direct proof that no unambiguous CFG can generate M .

A still stronger phenomenon

For the standard three-block language

$$L = \{a^n b^n c^k : n, k \geq 1\} \cup \{a^i b^n c^n : i, n \geq 1\},$$

Ogden showed that L^* has **unbounded inherent ambiguity**: in every CFG for L^* , the number of parses cannot be bounded by any fixed constant over all words.

Bijective renaming of terminals and reversal preserve inherent ambiguity, producing infinitely many equivalent variants. No regular language is inherently ambiguous because a DFA yields an unambiguous right-linear grammar.

Maurer's direct proof Ogden's strengthened result

Maurer's Example and Unbounded Ambiguity

A structurally different inherently ambiguous language is

$$M = \{a^n b^m a^r b^m : n, m, r \geq 1\} \\ \cup \{a^n b^m a^n b^s : n, m, s \geq 1\}.$$

The first component matches the two b -blocks; the second matches the two a -blocks. Their shared words include $(a^n b^m)^2$. Maurer gave a direct proof that no unambiguous CFG can generate M .

A still stronger phenomenon

For the standard three-block language

$$L = \{a^n b^n c^k : n, k \geq 1\} \cup \{a^i b^n c^n : i, n \geq 1\},$$

Ogden showed that L^* has **unbounded inherent ambiguity**: in every CFG for L^* , the number of parses cannot be bounded by any fixed constant over all words.

Bijjective renaming of terminals and reversal preserve inherent ambiguity, producing infinitely many equivalent variants. No regular language is inherently ambiguous because a DFA yields an unambiguous right-linear grammar.

Maurer's direct proof Ogden's strengthened result

Maurer's Example and Unbounded Ambiguity

A structurally different inherently ambiguous language is

$$M = \{a^n b^m a^r b^m : n, m, r \geq 1\} \\ \cup \{a^n b^m a^n b^s : n, m, s \geq 1\}.$$

The first component matches the two b -blocks; the second matches the two a -blocks. Their shared words include $(a^n b^m)^2$. Maurer gave a direct proof that no unambiguous CFG can generate M .

A still stronger phenomenon

For the standard three-block language

$$L = \{a^n b^n c^k : n, k \geq 1\} \cup \{a^i b^n c^n : i, n \geq 1\},$$

Ogden showed that L^* has **unbounded inherent ambiguity**: in every CFG for L^* , the number of parses cannot be bounded by any fixed constant over all words.

Bijective renaming of terminals and reversal preserve inherent ambiguity, producing infinitely many equivalent variants. No regular language is inherently ambiguous because a DFA yields an unambiguous right-linear grammar.

Maurer's direct proof Ogden's strengthened result

Challenge 1: Build a Union Language Carefully

Problem

Construct a CFG for

$$L = \{a^i b^j c^k : i = j \text{ or } j = k, i, j, k \geq 1\}.$$

Split the condition into two CFLs. One grammar is

$$\begin{aligned} S &\rightarrow X \mid Y, \\ X &\rightarrow AC, & A &\rightarrow aAb \mid ab, & C &\rightarrow cC \mid c, \\ Y &\rightarrow DB, & D &\rightarrow aD \mid a, & B &\rightarrow bBc \mid bc. \end{aligned}$$

X generates $a^n b^n c^k$ and Y generates $a^i b^n c^n$, so the union generates exactly L .
Every $a^n b^n c^n$ can be derived through both branches. This proves the displayed grammar ambiguous. The stronger claim that L is inherently ambiguous requires a theorem about the language, not merely these two derivations.

Challenge 1: Build a Union Language Carefully

Problem

Construct a CFG for

$$L = \{a^i b^j c^k : i = j \text{ or } j = k, i, j, k \geq 1\}.$$

Split the condition into two CFLs. One grammar is

$$\begin{aligned} S &\rightarrow X \mid Y, \\ X &\rightarrow AC, & A &\rightarrow aAb \mid ab, & C &\rightarrow cC \mid c, \\ Y &\rightarrow DB, & D &\rightarrow aD \mid a, & B &\rightarrow bBc \mid bc. \end{aligned}$$

X generates $a^n b^n c^k$ and Y generates $a^i b^n c^n$, so the union generates exactly L . Every $a^n b^n c^n$ can be derived through both branches. This proves the displayed grammar ambiguous. The stronger claim that L is inherently ambiguous requires a theorem about the language, not merely these two derivations.

Challenge 2: Equal Counts in Any Order

Problem

Show that

$$E = \{w \in \{a, b, c\}^* : |w|_a = |w|_b = |w|_c\}$$

is not context-free.

Assume E is context-free and intersect it with the regular language $a^*b^*c^*$. Closure under intersection with regular languages would make the result context-free. But

$$E \cap a^*b^*c^* = \{a^n b^n c^n : n \geq 0\},$$

which is not context-free. This contradiction proves that E is not a CFL.

Why this proof is efficient

Direct pumping must manage many symbol orders. The regular filter isolates the canonical ordered sublanguage where the known obstruction is visible.

Challenge 2: Equal Counts in Any Order

Problem

Show that

$$E = \{w \in \{a, b, c\}^* : |w|_a = |w|_b = |w|_c\}$$

is not context-free.

Assume E is context-free and intersect it with the regular language $a^*b^*c^*$. Closure under intersection with regular languages would make the result context-free. But

$$E \cap a^*b^*c^* = \{a^n b^n c^n : n \geq 0\},$$

which is not context-free. This contradiction proves that E is not a CFL.

Why this proof is efficient

Direct pumping must manage many symbol orders. The regular filter isolates the canonical ordered sublanguage where the known obstruction is visible.

Challenge 3: A Short Witness for Nonemptiness

Problem

An ε -free CNF grammar has k variables. Prove that if its language is nonempty, it contains a word of length at most 2^{k-1} .

Choose a shortest generated word w and a parse tree for it. If some variable A occurs twice on one root-to-leaf path, the upper A derives a context containing the lower A . Replacing the upper subtree by the lower subtree removes at least one productive side branch and yields a shorter terminal word, contradicting the choice of w .

Hence each path contains at most k variable nodes. A binary tree with at most k variable levels has at most 2^{k-1} terminal leaves. Since the yield length equals the number of terminal leaves, $|w| \leq 2^{k-1}$.

Challenge 3: A Short Witness for Nonemptiness

Problem

An ε -free CNF grammar has k variables. Prove that if its language is nonempty, it contains a word of length at most 2^{k-1} .

Choose a shortest generated word w and a parse tree for it. If some variable A occurs twice on one root-to-leaf path, the upper A derives a context containing the lower A . Replacing the upper subtree by the lower subtree removes at least one productive side branch and yields a shorter terminal word, contradicting the choice of w . Hence each path contains at most k variable nodes. A binary tree with at most k variable levels has at most 2^{k-1} terminal leaves. Since the yield length equals the number of terminal leaves, $|w| \leq 2^{k-1}$.

Challenge 4: Decide Whether a CFL Is Infinite

Problem

Give a graph-based finiteness test for a CFG.

First remove useless symbols and convert the grammar to CNF, treating ε separately. Build a dependency graph with an edge $A \rightarrow B$ whenever a rule for A contains B .

Criterion

The reduced language is infinite exactly when the dependency graph has a directed cycle reachable from the start variable.

In a reduced CNF grammar, every side variable on such a recursive cycle can derive a nonempty terminal word. Repeating the cycle therefore increases the yield length and produces infinitely many words. If there is no reachable cycle, derivation-tree height is bounded, so only finitely many parse trees and terminal words are possible.

Challenge 4: Decide Whether a CFL Is Infinite

Problem

Give a graph-based finiteness test for a CFG.

First remove useless symbols and convert the grammar to CNF, treating ε separately. Build a dependency graph with an edge $A \rightarrow B$ whenever a rule for A contains B .

Criterion

The reduced language is infinite exactly when the dependency graph has a directed cycle reachable from the start variable.

In a reduced CNF grammar, every side variable on such a recursive cycle can derive a nonempty terminal word. Repeating the cycle therefore increases the yield length and produces infinitely many words. If there is no reachable cycle, derivation-tree height is bounded, so only finitely many parse trees and terminal words are possible.

Challenge 5: Quotient by a Fixed Suffix

Problem

If L is context-free and u is a fixed word, prove that

$$L/\{u\} = \{x : xu \in L\}$$

is context-free.

Let an NPDA P recognize L . Construct P_u that reads the supplied input x exactly as P does. After the input ends, P_u enters a finite chain of control states that simulates P on the fixed symbols of u using ε -moves, while retaining the same stack.

The new machine accepts precisely when some computation of P accepts xu .

Therefore $L/\{u\}$ is context-free.

Generalization

CFLs are closed under left and right quotient with regular languages. The fixed-word construction is the simplest visible case of that theorem.

Challenge 5: Quotient by a Fixed Suffix

Problem

If L is context-free and u is a fixed word, prove that

$$L/\{u\} = \{x : xu \in L\}$$

is context-free.

Let an NPDA P recognize L . Construct P_u that reads the supplied input x exactly as P does. After the input ends, P_u enters a finite chain of control states that simulates P on the fixed symbols of u using ε -moves, while retaining the same stack.

The new machine accepts precisely when some computation of P accepts xu .

Therefore $L/\{u\}$ is context-free.

Generalization

CFLs are closed under left and right quotient with regular languages. The fixed-word construction is the simplest visible case of that theorem.

Challenge 6: CFL or Not? Use Structure First

Classify the following languages and justify each answer.

- ① $L_1 = \{a^n b^m c^m d^n : n, m \geq 0\}$ is a CFL; use $S \rightarrow aSd \mid T$ and $T \rightarrow bTc \mid \epsilon$.
- ② $L_2 = \{a^n b^m c^n d^m : n, m \geq 0\}$ is not a CFL. For $a^p b^p c^p d^p$, the window vwx touches at most two adjacent blocks. Pumping changes a block without its distant matching partner, so either $|a| = |c|$ or $|b| = |d|$ fails.
- ③ $L_3 = \{wcw^R : w \in \{0, 1\}^*\}$ is a DCFL; push before c and pop while matching afterward.
- ④ $L_4 = \{ww : w \in \{0, 1\}^*\}$ is not a CFL; the second half must copy the first in the same order rather than reverse stack order.

Exam strategy

Before choosing a proof tool, ask whether the dependencies are nested, separated by a marker, or crossed. That structural diagnosis often determines the correct construction or contradiction.

Challenge 6: CFL or Not? Use Structure First

Classify the following languages and justify each answer.

- ❶ $L_1 = \{a^n b^m c^m d^n : n, m \geq 0\}$ is a CFL; use $S \rightarrow aSd \mid T$ and $T \rightarrow bTc \mid \varepsilon$.
- ❷ $L_2 = \{a^n b^m c^n d^m : n, m \geq 0\}$ is not a CFL. For $a^p b^p c^p d^p$, the window vwx touches at most two adjacent blocks. Pumping changes a block without its distant matching partner, so either $|a| = |c|$ or $|b| = |d|$ fails.
- ❸ $L_3 = \{wcw^R : w \in \{0, 1\}^*\}$ is a DCFL; push before c and pop while matching afterward.
- ❹ $L_4 = \{ww : w \in \{0, 1\}^*\}$ is not a CFL; the second half must copy the first in the same order rather than reverse stack order.

Exam strategy

Before choosing a proof tool, ask whether the dependencies are nested, separated by a marker, or crossed. That structural diagnosis often determines the correct construction or contradiction.

Challenge 6: CFL or Not? Use Structure First

Classify the following languages and justify each answer.

- ① $L_1 = \{a^n b^m c^m d^n : n, m \geq 0\}$ is a CFL; use $S \rightarrow aSd \mid T$ and $T \rightarrow bTc \mid \varepsilon$.
- ② $L_2 = \{a^n b^m c^n d^m : n, m \geq 0\}$ is not a CFL. For $a^p b^p c^p d^p$, the window vwx touches at most two adjacent blocks. Pumping changes a block without its distant matching partner, so either $|a| = |c|$ or $|b| = |d|$ fails.
- ③ $L_3 = \{wcw^R : w \in \{0, 1\}^*\}$ is a DCFL; push before c and pop while matching afterward.
- ④ $L_4 = \{ww : w \in \{0, 1\}^*\}$ is not a CFL; the second half must copy the first in the same order rather than reverse stack order.

Exam strategy

Before choosing a proof tool, ask whether the dependencies are nested, separated by a marker, or crossed. That structural diagnosis often determines the correct construction or contradiction.

Challenge 6: CFL or Not? Use Structure First

Classify the following languages and justify each answer.

- ① $L_1 = \{a^n b^m c^m d^n : n, m \geq 0\}$ is a CFL; use $S \rightarrow aSd \mid T$ and $T \rightarrow bTc \mid \varepsilon$.
- ② $L_2 = \{a^n b^m c^n d^m : n, m \geq 0\}$ is not a CFL. For $a^p b^p c^p d^p$, the window vwx touches at most two adjacent blocks. Pumping changes a block without its distant matching partner, so either $|a| = |c|$ or $|b| = |d|$ fails.
- ③ $L_3 = \{wcw^R : w \in \{0, 1\}^*\}$ is a DCFL; push before c and pop while matching afterward.
- ④ $L_4 = \{ww : w \in \{0, 1\}^*\}$ is not a CFL; the second half must copy the first in the same order rather than reverse stack order.

Exam strategy

Before choosing a proof tool, ask whether the dependencies are nested, separated by a marker, or crossed. That structural diagnosis often determines the correct construction or contradiction.

About the Book-Style Challenges

The preceding exercises are newly phrased and solved from recurring themes in standard formal-language texts rather than copied verbatim:

- grammar construction by union and structural decomposition;
- regular filtering to expose a known non-CFL;
- CNF parse-tree bounds and grammar finiteness;
- quotient closure and PDA simulation; and
- nested versus crossed stack dependencies.

Recommended workflow

Try a grammar or PDA construction first when obligations are nested. Use closure with a regular filter when it isolates a standard hard language. Reserve pumping or Ogden's lemma for cases where constructive approaches fail and every legal decomposition can be controlled.

About the Book-Style Challenges

The preceding exercises are newly phrased and solved from recurring themes in standard formal-language texts rather than copied verbatim:

- grammar construction by union and structural decomposition;
- regular filtering to expose a known non-CFL;
- CNF parse-tree bounds and grammar finiteness;
- quotient closure and PDA simulation; and
- nested versus crossed stack dependencies.

Recommended workflow

Try a grammar or PDA construction first when obligations are nested. Use closure with a regular filter when it isolates a standard hard language. Reserve pumping or Ogden's lemma for cases where constructive approaches fail and every legal decomposition can be controlled.

Appendix Guide: Optional Theory Slides

For a shorter first course, retain the formal definitions and omit or move the following enrichment slides to reading material:

- *Reusable Grammar-Design Patterns II*
- *Parse Trees and Abstract Syntax Trees*
- *The Same Ambiguity as Leftmost Derivations*
- *The Dangling-Else Ambiguity*
- *Grammar Ambiguity Versus Inherent Ambiguity*
- *Understanding an Inherently Ambiguous Language*
- *Predictive Recognition for an s -Grammar*
- *What Fails the s -Grammar Test?*
- *Meaning of Equivalent Normal Form*
- *A Systematic GNF Conversion Outline*
- *GNF Versus an s -Grammar*
- *Other Useful Grammar Forms and Restrictions*
- *Important CKY/CYK Extensions*
- *CYK in Practice and Important Alternatives*

These slides add depth but are not prerequisites for the core CFG–PDA equivalence normally taught next.

Appendix Guide: Optional Examples and Review

Keep as the minimum conceptual spine

CFG definition; derivations; complete and partial derivation trees; one ambiguity example; simplification procedures; CNF; the core CYK recurrence; CFL closure/nonclosure; the pumping theorem and game; and one complete non-CFL proof.

Select rather than present all

- Keep *Worked CNF Conversion* and either the worked CYK table or the CYK exercise, not necessarily both.
- Keep pumping Example 1 and choose only one of Examples 2–7 based on the examination level.
- Choose either *Worked Closure Examples* or *Filtering a CFL with a Regular Language*.
- Move all hint slides to an appendix or distribute them after class.
- Omit BNF/EBNF and compiler-pipeline slides if those topics are already covered in a compiler course.

References: Advanced CFL Results



Rohit J. Parikh,
On Context-Free Languages,
Journal of the ACM, 13(4), 1966.



William Ogden,
A Helpful Result for Proving Inherent Ambiguity, 1968.



Herman A. Maurer,
A Direct Proof of the Inherent Ambiguity of a Simple Context-Free Language, 1969.



Noam Chomsky and Marcel-Paul Schützenberger,
The Algebraic Theory of Context-Free Languages, 1959.



William Ogden, Rockford J. Ross, and Karl Winklmann,
An Interchange Lemma for Context-Free Languages, 1985.



Géraud Sénizergues,
The Equivalence Problem for Deterministic Pushdown Automata Is Decidable, 1997.

References: Parsing Algorithms

 Daniel Jurafsky and James H. Martin,
Speech and Language Processing, 3rd-edition draft,
chapters on constituency grammars, PCFGs, and CKY parsing.

 Jay Earley,
An Efficient Context-Free Parsing Algorithm,
Communications of the ACM, 13(2), 1970.

 Leslie G. Valiant,
General Context-Free Recognition in Less than Cubic Time,
Journal of Computer and System Sciences, 10(2), 1975.

 Masaru Tomita,
An Efficient Augmented-Context-Free Parsing Algorithm,
Computational Linguistics, 13(1-2), 1987.

References and Suggested Reading



Peter Linz and Susan H. Rodger,
An Introduction to Formal Languages and Automata, 7th ed.,
Jones & Bartlett Learning, 2023.



Michael Sipser,
Introduction to the Theory of Computation, 3rd ed.,
Cengage Learning, 2013.



John E. Hopcroft, Rajeev Motwani, and Jeffrey D. Ullman,
Introduction to Automata Theory, Languages, and Computation, 3rd ed.,
Pearson, 2006.



Jeffrey Shallit,
A Second Course in Formal Languages and Automata Theory,
Cambridge University Press, 2009.